

VIETNAM NATIONAL UNIVERSITY OF HOCHIMINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



SOFTWARE ENGINEERING
IT076IU

FINAL REPORT

Topic: AI shopping system

By Group 01 – Member List

1	Nguyễn Thành Nam	ITCSIU22311	Project Manager
2	Đào Hữu Hoài	ITCSIU23054	Front-end Leader
3	Hồ Thiên Trường	ITCSIU22306	Member
4	Nguyễn Trí Dũng	ITCSIU22242	Member
5	Lê Thiên Doanh	ITCSIU22237	Back-end Leader
6	Huỳnh Hữu Nghĩa	ITCSIU21207	Member
7	Nguyễn Hoàng Bảo Khang	ITCSIU22234	Member
8	Bành Vĩnh Thuận	ITITI21323	Report Leader
9	Phạm Xuân Trường	ITITI21336	Member
10	Hà Tấn Tài	ITCSIU21227	Member

Instructor: Assoc. Prof. Nguyen Thi Thuy Loan

Link GitHub: [Group 1 - AI Shopping System](#)

Link Drive: [SE Project \(2025\)](#)

TABLE OF CONTENTS

LIST OF FIGURES	vi
LIST OF TABLES	vii
ABSTRACT	ix
CHAPTER 1	1
INTRODUCTION	1
1.1. Background	1
1.2. Problem Statement	1
1.3. Scope and Object.....	1
1.3.1. Scope	1
1.3.2. Objectives	2
1.4. Customer Requirements	3
1.5. Report Structure	3
CHAPTER 2	4
PROJECT TIMELINE AND IN-CHARGE TABLE	4
2.1. Project Timeline	4
2.1.1. Project Timeline	4
2.1.2. Sprint Backlog.....	5
2.2. In-Charge Table.....	7
CHAPTER 3	8
METHODOLOGY	8
3.1. Overview.....	8
3.2. User Requirement Analysis	9
3.2.1. Requirement Register (Functional and Non-functional).....	9
3.2.2. Resource Register (Hardware, software)	11
3.2.2.1. Hardware resource	11
3.2.2.2. Software resource	11
3.2.3. Risk register.....	12
3.3. Requirement Specifications	14
3.3.1. Use Case Diagram	14
3.3.2. Use Case Descriptions	15
3.4. System Design	31

3.4.1.	<i>System Architecture</i>	31
3.4.2.	<i>Entity Relationship Diagram</i>	32
3.4.3.	<i>Class Diagram</i>	34
3.4.4.	<i>Sequence Diagram</i>	36
3.4.4.1.	<i>Suggest Personal</i>	36
3.4.4.2.	<i>Payment Process</i>	38
3.4.4.3.	<i>Leave Feedback</i>	40
3.4.4.4.	<i>3D Product View Request</i>	42
3.4.5.	<i>User Interface Design</i>	43
CHAPTER 4		46
IMPLEMENT AND RESULT		46
4.1.	<i>Implement</i>	46
4.1.1.	<i>Front-end</i>	46
4.1.1.1.	<i>Language</i>	46
4.1.1.2.	<i>Framework</i>	46
4.1.2.	<i>Back-end</i>	47
4.1.2.1.	<i>Environment</i>	47
4.1.2.2.	<i>Framework</i>	47
4.1.2.3.	<i>Database</i>	47
4.2.	<i>Results of System's Architecture</i>	48
4.2.1.	<i>Backend Overview</i>	48
4.2.2.	<i>Frontend Overview</i>	49
CHAPTER 5		50
DISCUSSION, TESTING AND EVALUATION		50
5.1.	<i>Discussion</i>	50
5.2.	<i>Testing methodologies employed</i>	50
5.2.1.	<i>Unit tests</i>	50
5.2.2.	<i>Integration tests</i>	51
5.2.3.	<i>System tests</i>	51
5.3.	<i>Test Cases</i>	52
5.3.1.	<i>Front-end</i>	52
5.3.2.	<i>Back-end</i>	56

5.4. Comparison	59
5.5. Evaluation	59
CHAPTER 6	60
CONCLUSION AND FEATURE WORK	60
6.1 Conclusion.....	60
6.2 Future work	60
REFERENCES	62
APPENDIX.....	63
Appendix A. Security Measures	63
1. Authentication	63
2. Authorization	63
3. Data Protection	63
4. Payment Security.....	63
Appendix B. User Feedback Analysis.....	65
1. Positive Feedback Highlights.....	65
2. Negative Feedback Highlights	65
3. Action Items Based on Feedback	65
Appendix C. Meeting Minutes	67
Appendix D. Contact Information	68

LIST OF FIGURES

<i>Figure 1.1: Gantt Chart</i>	<i>4</i>
<i>Figure 2.1: Use case diagram</i>	<i>14</i>
<i>Figure 2.3: Entity Relationship Diagram.....</i>	<i>32</i>
<i>Figure 2.4: Table Schema</i>	<i>33</i>
<i>Figure 2.5: Class Diagram.....</i>	<i>34</i>
<i>Figure 2.6: Sequence Diagram _Suggest Personal</i>	<i>36</i>
<i>Figure 2.7: Sequence Diagram _Payment Process.....</i>	<i>38</i>
<i>Figure 2.8: Sequence Diagram _Leave feedback</i>	<i>40</i>
<i>Figure 2.9: Sequence Diagram _3D Product View Request.....</i>	<i>42</i>
<i>Figure 2.10: Default Header.....</i>	<i>43</i>
<i>Figure 2.11: Logged in Header.....</i>	<i>43</i>
<i>Figure 2.12: Sub navigation bar.....</i>	<i>43</i>
<i>Figure 2.13: Log in page.....</i>	<i>44</i>
<i>Figure 2.14: Register info</i>	<i>44</i>
<i>Figure 2.15: Confirm email</i>	<i>44</i>
<i>Figure 2.16: Checkout</i>	<i>44</i>
<i>Figure 2.17: Change password.....</i>	<i>44</i>
<i>Figure 2.18: Order</i>	<i>44</i>
<i>Figure 2.19: List product</i>	<i>45</i>
<i>Figure 2.20: Product detail page</i>	<i>45</i>
<i>Figure 2.21: Cart page</i>	<i>45</i>
<i>Figure 2.22: Home page</i>	<i>45</i>

LIST OF TABLES

<i>Table 1.1: Sprint 1</i>	<i>5</i>
<i>Table 1.2: Sprint 2</i>	<i>6</i>
<i>Table 1.3: Sprint 3</i>	<i>6</i>
<i>Table 1.4: Roles and Responsibilities</i>	<i>7</i>
<i>Table 2.1: Functional and Non-functional requirements.....</i>	<i>9</i>
<i>Table 2.2: Hardware resources</i>	<i>11</i>
<i>Table 2.3: Software resource.....</i>	<i>11</i>
<i>Table 2.4: Risk Register.....</i>	<i>12</i>
<i>Table 2.5: UC1_Sign Up.....</i>	<i>15</i>
<i>Table 2.6: UC2_Log In.....</i>	<i>16</i>
<i>Table 2.7: UC3_View 3D Jewelry</i>	<i>17</i>
<i>Table 2.8: UC4_Search Product</i>	<i>18</i>
<i>Table 2.9: UC5_Custom</i>	<i>19</i>
<i>Table 2.10: UC6_View Items.....</i>	<i>20</i>
<i>Table 2.11: UC7_Add to Reminder</i>	<i>21</i>
<i>Table 2.12: UC8_Pay.....</i>	<i>22</i>
<i>Table 2.13: UC9_Checkout</i>	<i>23</i>
<i>Table 2.14: UC10_View Cart.....</i>	<i>24</i>
<i>Table 2.15: UC11_View Orders.....</i>	<i>25</i>
<i>Table 2.16: UC12_Leave Feedback/Review</i>	<i>26</i>
<i>Table 2.17: UC13_Add to Cart.....</i>	<i>27</i>
<i>Table 2.18: UC14_Confirm Registration Email.....</i>	<i>28</i>
<i>Table 2.19: UC15_Confirm Registration Email.....</i>	<i>29</i>
<i>Table 2.20: UC16_Answer Product's Related Question.....</i>	<i>30</i>
<i>Table 2.21: Schema description</i>	<i>33</i>
<i>Table 3.1: Unit test applications.....</i>	<i>50</i>
<i>Table 3.2: Integration test applications</i>	<i>51</i>
<i>Table 3.3: System test applications</i>	<i>51</i>
<i>Table 3.4: Test Case 1 Information.....</i>	<i>52</i>
<i>Table 3.5: Test Case 2 Information.....</i>	<i>52</i>
<i>Table 3.6: Test Case 3 Information.....</i>	<i>53</i>

<i>Table 3.7: Test Case 4 Information.....</i>	<i>54</i>
<i>Table 3.8: Test Case 5 Information.....</i>	<i>54</i>
<i>Table 3.9: Test Case 6 Information.....</i>	<i>55</i>
<i>Table 3.10: Test Case 7 Information.....</i>	<i>56</i>
<i>Table 3.11: Test Case 8 Information</i>	<i>56</i>
<i>Table 3.12: Test Case 9 Information.....</i>	<i>57</i>
<i>Table 3.13: Test Case 10 Information.....</i>	<i>57</i>
<i>Table 3.14: Test Case 11 Information</i>	<i>58</i>
<i>Table 3.15: Test Case 12 Information.....</i>	<i>58</i>

ABSTRACT

The AI-Powered Jewelry Shopping System is an intelligent web platform designed to enhance the online jewelry shopping experience by integrating artificial intelligence and advanced visualization technologies. This system provides users with AI-driven recommendations, real-time chatbot assistance, and an interactive 3D jewelry customization tool. By allowing customers to design, preview, and personalize jewelry pieces before purchasing, the platform ensures a seamless and engaging shopping journey.

The project aims to improve personalization, transparency, and user convenience in the jewelry industry. Automated pricing calculations based on materials, design complexity and promotional offers provide clear cost estimations, while secure payment processing ensures a reliable transaction experience. Additionally, virtual trials help minimize uncertainties, increasing customer confidence and reducing product returns. By leveraging AI and e-commerce innovations, this system seeks to revolutionize jewelry shopping, making it more intuitive, efficient, and accessible for a diverse range of users.

CHAPTER 1

INTRODUCTION

1.1. Background

In the digital age, traditional online shopping often lacks personalization and interactive product experiences, especially in the jewelry sector. Our project aims to enhance the online jewelry shopping journey by integrating AI and immersive 3D technology.

The platform features AI-driven product recommendations, a real-time chatbot for instant support, and 360° jewelry previews, making confident purchase decisions without visiting a store.

By combining smart technology with user-friendly design, the system delivers a seamless, engaging, and personalized shopping experience tailored to modern consumer needs.

1.2. Problem Statement

Traditional e-commerce platforms often lack personalization, interactivity, and precision in product evaluation. To overcome these limitations, the AI Multi-Agent Shopping System introduces:

- AI-driven product recommendations based on user behavior and purchase history.
- A real-time AI chatbot for instant and context-aware support.
- 3D interactive previews, allowing users to have a 360° look of jewelry before purchasing.

By integrating AI and immersive technologies, our system enhances decision-making, minimizes uncertainty, and delivers a seamless, engaging shopping experience.

1.3. Scope and Object

1.3.1. Scope

Our project focuses on developing an AI-powered jewelry customization and shopping system, featuring:

- User Authentication and Profiles: Account registration, profile management, saved designs, and purchase history.
- Jewelry Customization: Interactive tool for selecting gemstones, metals, engravings, and sizes, with AI-driven suggestions based on preferences and trends.
- Real-Time 3D Visualization: High-quality rendering for users to preview their designs from different angles before purchase.
- Automated Pricing: A system that calculates costs based on materials, design complexity, and promotional offers to provide transparent pricing.
- E-commerce and Payment Integration: Secure payment processing, multiple payment options, invoicing, and order tracking.
- Third-Party Integration: Collaboration with jewelry manufacturers and retailers for seamless production and delivery.
- Continuous Improvement: Regular updates and enhancements based on customer feedback and market trends.

1.3.2. Objectives

Our project aims to create an intuitive and efficient AI-powered jewelry customization and shopping system by achieving the following objectives:

- Ease of Use: Provide an intuitive platform for designing and purchasing custom jewelry, simplifying the process for users.
- Accessibility: Ensure a user-friendly experience for both beginners and professionals, with real-time error validation.
- Personalization: Offer AI-driven recommendations based on user preferences and market trends.

- Transparency: Implement automated pricing for clear, upfront cost estimation based on materials, design complexity and promotional offers.
- Efficiency: Optimize manufacturing by generating precise design specifications, reducing waste and errors.

1.4. Customer Requirements

The development of this AI Shopping System aligns with the expectations of our industry client, who highlights the importance of personalization, accuracy, and convenience in the online shopping journey. There is a strong demand for a platform that integrates AI-driven recommendations, real-time customer support, and interactive 3D previews, allowing users to explore jewelry in detail.

1.5. Report Structure

This report is organized into six chapters. Chapter 1 introduces the background, problem statement, scope and objectives, customer requirements, and provides an overview of the report structure. Chapter 2 outlines the project timeline and the responsibilities assigned to each team member through the in-charge table. Chapter 3 describes the methodology adopted, including requirement analysis, system design, and specification details. Chapter 4 presents the implementation process, highlighting the technologies used and showcasing the resulting system architecture. Chapter 5 discusses the overall development, testing strategies, comparison with initial objectives, and evaluation of system performance. Finally, Chapter 6 concludes the report and suggests potential directions for future improvements..

CHAPTER 2

PROJECT TIMELINE AND IN-CHARGE TABLE

2.1. Project Timeline

2.1.1. Project Timeline

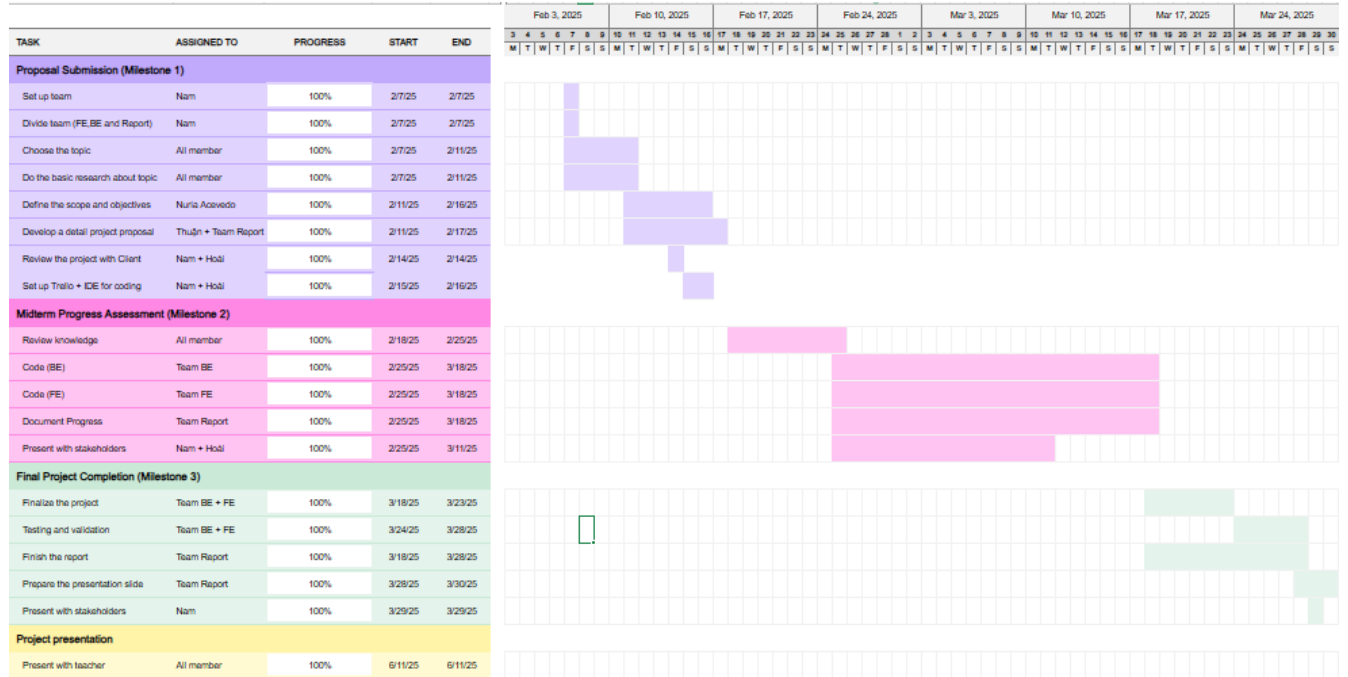


Figure 1.1: Gantt Chart

- **Research Phase:** Our initial focus is on in-depth research to identify techniques, frameworks, and processes that align optimally with the goals and objectives of our project. Our objective is to make well-informed decisions that will result in maximum project satisfaction and the achievement of expected outcomes.
- **Implementation Phase:** We plan to adopt Scrum, an Agile framework, during the development stage. This approach allows us to divide project features into multiple sprints, each delivering tangible and usable results. The duration and budget for each sprint will be determined based on the complexity of the tasks and any evolving requirements.

- **Midterm Progress Evaluation:** Regular progress updates will be provided, showcasing completed features and detailing the ongoing sprint's advancements. This transparent approach aims to offer you valuable insights into the development process, enabling you to make informed decisions regarding the project's continuation.
- **Enhancement Phase (Testing + Implementation):** Stakeholder’s feedback on completed functionalities is highly valued. We are committed to using their input to enhance the product's alignment with our business needs and improve operational efficiency. Additionally, we prioritize incorporating stakeholder feedback into our testing processes to ensure the product meets quality standards, performs reliably under various scenarios, and addresses any identified issues effectively.

2.1.2. Sprint Backlog

Table 1.1: Sprint 1

SPRINT 1 (17/02/2025 - 10/03/2025)			
User Story: As a User, I want to sign up and log in to the platform so that I can access personalized features and start shopping for jewelry.		Goal: Enable Users to create accounts and log in securely, laying the foundation for authenticated interactions with the system.	
Definition of Done: - Code is complete, reviewed, and free of critical bugs. - Registration and login are fully functional and securely integrated with the backend. - User data is securely stored in the database with proper encryption. - Acceptance criteria are met: Users can sign up with valid email and password, log in, and receive appropriate error messages for invalid inputs. - Unit tests for registration and login pass with at least 90% coverage. - UI displays user-friendly feedback for errors (e.g., duplicate email, wrong password).			
No.	Task	Assignee	Deadline
1	UI Building (Account Registration)	Hoài	08/03/2025
2	Setup database	Hữu Nghĩa	08/03/2025
3	Implement Login & Registration Fetching	Hữu Nghĩa	08/03/2025

Table 1.2: Sprint 2

SPRINT 2 (10/03/2025 - 31/03/2025)			
User Story: As a User, I want to browse the homepage and search for jewelry products so that I can explore available items and find what I like.		Goal: Deliver a functional homepage with search capabilities, enabling Users to explore jewelry products and interact with the system intuitively.	
Definition of Done: - Homepage UI is fully designed, responsive, and consistent across devices. - Search functionality retrieves relevant products from the database based on keywords. - Acceptance criteria are met: Users can access the homepage, search for products (e.g., "gold ring"), and view results with basic filters (e.g., price, category). - API integration for product search is stable and handles errors gracefully. - Unit and integration tests for search functionality pass. - Loading times for homepage and search results are under 2 seconds.			
No.	Task	Assignee	Deadline
1	UI Rebuilding (Homescreen Design)	Hữu Hoài, Trường	31/03/2025
2	Homescreen Functionality Implementation	Hữu Nghĩa	31/03/2025
3	Testing UI & Homescreen Features	Khang	31/03/2025

Table 1.3: Sprint 3

SPRINT 3 (15/04/2025 - 04/05/2025)	
User Story: As a Customer, I want to customize jewelry and preview it in 3D so that I can visualize my design before purchasing.	Goal: Enable Customers to use an interactive 3D jewelry customization tool, supporting personalization and enhancing purchase confidence.
Definition of Done: - 3D customization tool is fully functional, allowing customers to select materials, gemstones, and sizes.	

- Real-time 3D previews update dynamically based on user selections. - Pricing updates automatically based on customization choices. - Acceptance criteria are met: Customers can customize a ring (e.g., select gold and diamond), view it in 3D, and see an accurate price. - Integration tests for 3D rendering and pricing logic pass. - Feature is demoed to stakeholders and feedback is incorporated.			
No.	Task	Assignee	Deadline
1	Develop 3D Jewelry Customization Tool	Thiên Doanh	04/05/2025
2	Integrate Real-Time 3D Jewelry Previews	Thiên Doanh	04/05/2025
3	Backend Integration (Pricing, Customization)	Hữu Nghĩa	04/05/2025
4	Testing of 3D Customization and Pricing Features	Tài, Xuân Trường	04/05/2025

2.2. In-Charge Table

Table 1.4: Roles and Responsibilities

Name	Responsibility	Contribution
Nguyễn Thành Nam	Project Manager	10%
Đào Hữu Hoài	Front-end Leader	11%
Hồ Thiên Trường	Front-end member	10%
Nguyễn Trí Dũng	Front-end member	9%
Lê Thiên Doanh	Back-end Leader (AI)	10%
Huỳnh Hữu Nghĩa	Back-end member (Back-end)	11%
Nguyễn Hoàng Bảo Khang	Backend member	9%
Bành Vĩnh Thuận	Report Leader	12%
Phạm Xuân Trường	Report member	8%
Hà Tấn Tài	Report member	10%

CHAPTER 3

METHODOLOGY

3.1. Overview

The methodology for developing the AI Shopping System begins with a comprehensive analysis of user requirements to ensure the system aligns with customer expectations and retail trends. Both functional requirements, such as product recommendation and viewing product 3D, and non-functional requirements, like system scalability and response time, are identified. A resource register is created to document the required personnel, AI tools, and computational resources, while a risk register is prepared to anticipate challenges such as algorithm bias, data privacy concerns, and integration issues.

Subsequently, the requirement specifications are formalized using use case diagrams and detailed descriptions to capture system interactions involving users, the AI engine, and the shopping database. This step ensures that all user flows—like personalized recommendations, virtual try-on, or smart search—are clearly defined.

Our project employs the Scrum Agile methodology, utilizing short sprints (e.g., 3-4 weeks) to deliver iterative increments, such as user authentication and 3D customization. Daily Scrum meetings ensure team alignment, while sprint planning and reviews with stakeholder feedback drive continuous improvement. This approach enhances flexibility, enables early issue resolution (e.g., AI inaccuracies), and fosters collaboration across front-end, back-end, and AI teams, aligning with the project's iterative development goals.

In the system design phase, the Entity-Relationship Diagram (ERD) is used to model the underlying database, including entities such as User, Product, Cart, Order, and AIProfile. The Class Diagram then defines object-oriented structures including methods for machine learning processing and

recommendation logic. Sequence Diagrams illustrate dynamic processes like paying for the product or generating real-time suggestions and viewing 3D product.

Database design is implemented by defining relational tables that support AI integration and user interactions. The User Interface Design (UID) phase focuses on creating intuitive, adaptive layouts using wireframes and prototypes for desktops and mobile platforms, ensuring seamless user experiences. This structured methodology supports the development of a robust, intelligent shopping system that enhances customer engagement through AI-powered features.

3.2. User Requirement Analysis

3.2.1. Requirement Register (Functional and Non-functional)

Table 2.1: Functional and Non-functional requirements

Req.ID	Requirement Name	Detailed Description	Type
001	User Registration and Login	Users can create accounts using email. Secure login system with password recovery and registration prompts for new users.	Functional Requirement
002	Jewelry Customization	Allows users to personalize jewelry with gemstone choices, metal types, ring sizes, and engravings.	Functional Requirement
003	Drag-and-Drop and AI Suggestion	Provides drag-and-drop design interface. AI recommends items based on user preferences, past purchases, and trending styles.	Functional Requirement
004	Real-time 3D Visualization	Offers high-quality 3D rendering that previews jewelry designs from various angles in real time.	Functional Requirement
005	Automated Pricing	Automatically calculates final pricing based on selected materials, customizations, and promotional offers.	Functional Requirement

006	Secure Payment and Checkout	Supports multiple secure payment methods including credit cards, e-wallets, with encrypted gateways.	Functional Requirement
007	Order Tracking	Allows customers to track their order status from confirmation through delivery.	Functional Requirement
008	Customer Feedback System	Enables users to review purchased products and give comments.	Functional Requirement
009	Scalability	System must handle large user traffic during peak shopping times using scalable cloud infrastructure.	Non-Functional Requirement
010	Performance	Smooth user experience with high-speed rendering, fast response times, and low latency in AI suggestions.	Non-Functional Requirement
011	Security and Compliance	Implements encryption for sensitive data, adheres to GDPR and PCI DSS for secure data and payment handling.	Non-Functional Requirement
012	Usability	Interface should be intuitive, responsive, and accessible across different devices and platforms.	Non-Functional Requirement
013	Reliability	Ensures system availability with minimal downtime and includes robust error handling and backup systems.	Non-Functional Requirement
014	Maintainability	Modular architecture and clean code structure for ease of updates. Includes full documentation for developers and system administrators.	Non-Functional Requirement
015	AI Efficiency	AI engines must process large datasets efficiently, ensuring fast, accurate personalization and recommendations.	Non-Functional Requirement

016	Cross-Platform Compatibility	System should function consistently on major browsers and both desktop and mobile environments.	Non-Functional Requirement
017	Environmental Responsibility	Promotes paperless digital receipts and eco-conscious packaging and delivery methods.	Non-Functional Requirement

3.2.2. Resource Register (Hardware, software)

3.2.2.1. Hardware resource

Table 2.2: Hardware resources

Items	Quantity	Description	Purpose	Price (\$)	Total Cost (\$)	Data needed
HP Pavilion 15	1	Mobile PC	Programming and Writing report	855	855	10/2/2025
Acer Predator Helios 300	1	Mobile PC	Backend Programming and Writing Report	1200	1200	14/2/2025
MSI GF63 Thin	1	MobilePC	Writing Report	1700	1700	10/2/2025
MacBook M3 Pro	1	Mobile PC	Backend Programming and Writing Report	2000	2000	14/2/2025
Total Cost: 5755 \$						

3.2.2.2. Software resource

Table 2.3: Software resource

Application	Quantity	Description	Purpose	Price (\$)	Total Cost (\$)	Data needed
Visual Studio Code	9	Code Editor	Programming Environment	Free	Free	10/2/2025
MongoDB	1	NoSQL Database as a service	Project Database	Free	Free	14/2/2025

Node.js	1	JavaScript runtime environment	Backend Programming	Free	Free	14/2/2025
React.js	1	Frontend JavaScript framework	Frontend Programming	Free	Free	10/2/2025
GitHub	1	Software Management Platform	Project storage & management	Free	Free	10/2/2025

3.2.3. Risk register

Table 2.4: Risk Register

Risk ID	Risk Category	Risk Title	Affect	Probability	Impact	Risk Response Plan
001	HR	Team Collaboration Breakdown	Miscommunication or poor coordination can slow development and reduce solution quality.	High	Medium	Applying direct communication between each member and resolve conflicts promptly.
002		Loss of Key Developers	Losing developers with AI or 3D expertise can delay core functionalities.	Low	High	Cross-train team members and document critical knowledge.
003	Requirements	Unclear or Changing Requirements	Frequent changes may cause scope creep and delay system integration.	High	Medium	Conduct thorough requirement analysis and enforce change control processes.

004	Management	Budget Overrun	High costs for AI models, 3D tech, or cloud services may exceed initial estimations.	Medium	High	Define budget buffers and monitor expenses regularly.
005	Schedule	Delays in Model Training or Testing	AI model training/testing may take longer, affecting deployment timeline.	Medium	High	Allocate buffer time and use parallel development when possible.
006	Quality	Inadequate Testing of AI & UX Flows	Poor testing may result in irrelevant recommendations or confusing 3D interactions.	Medium	High	Implement rigorous testing, including unit, integration, and user acceptance testing.
007		Poor 3D User Interface Design	Low-quality previews or laggy performance can drive users away.	Medium	High	Use UI/UX design reviews and conduct user feedback sessions regularly.
008	Scope	Undefined Technical Boundaries	Lack of clear limits on AI or 3D features may stretch development capacity.	Medium	High	Clearly define feature scope and set technical constraints early.
009	Cost	Unanticipated Integration Costs	Integrating real-time 3D with e-commerce systems may incur unforeseen expenses.	Medium	Medium	Conduct early feasibility studies and plan contingency budget.
010	Technology	AI Model Inaccuracy	Inaccurate recommendations reduce personalization and user trust.	Medium	High	Continuously train and validate AI models using real user data.

3.3. Requirement Specifications

3.3.1. Use Case Diagram

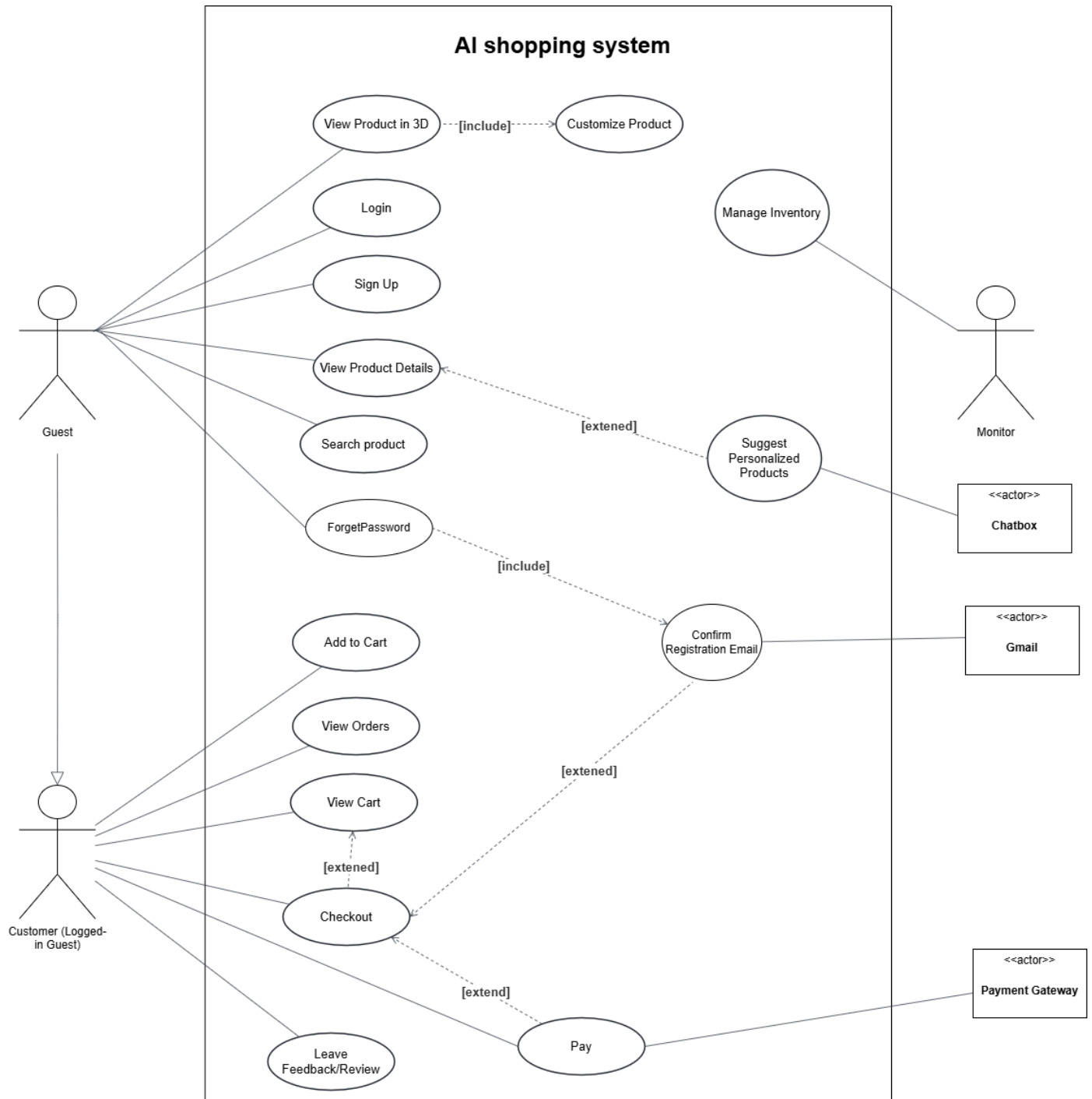


Figure 2.1: Use case diagram

3.3.2. Use Case Descriptions

0.1 Use Case 1: Sign Up

Name: Sign Up

Identifier: UC1

Inputs:

1. Personal information (Full Name, Email, Phone Number, etc.)
2. Account credentials (Username, Password)

Outputs:

1. Confirmation message [If successful]
2. Error message [If failed]

Basic Course:

Table 2.5: UC1_Sign Up

Actor: Guest	System
Access the registration page.	Display registration form.
Enter personal information and account credentials.	Validate the provided information.
Submit the registration form.	Store the user credentials and personal info. Display confirmation message (successful registration) or error message (failure).

Preconditions:

1. The user has not created an account yet.

Postconditions:

1. The user's account is created and stored in the system.

User Story:

As a user, I want to create an account so that I can start shopping on the platform.

0.2 Use Case 2: Log in

Name: Log In

Identifier: UC2

Inputs:

1. Username
2. Password

Outputs:

1. Welcome message [If successful]
2. Error message [If credentials are incorrect]

Basic Course:

Table 2.6: UC2_Log In

Actor: Guest	System
Navigate to the login page.	Display login form.
Enter username and password.	Validate the credentials.
Submit the login form.	Authenticate the user. If successful, display a welcome message and direct the user to the dashboard. If failed, show error message.

Preconditions:

1. The user has already created an account.

Postconditions:

1. The user is logged into the system and granted access to AI-powered jewelry shopping features, including personalized recommendations, 3D previews, and virtual try-on tools

User Story:

As a user, I want to log into my account so that I can access personalized jewelry recommendations and complete purchases easily.

0.3 Use Case 3: View Product in 3D

Name: View Product in 3D

Identifier: UC3

Inputs:

1. User clicks on the “View in 3D” button on a product detail page.

Outputs:

1. Display an interactive 3D model of the selected jewelry item.
2. Allow user to rotate, zoom, and inspect the item in 360 degrees.

Basic Course:

Table 2.7: UC3_View 3D Jewelry

Actor: Guest	System
User is on the item detail page.	Shows “View in 3D” option if model is available.
Clicks the “View in 3D” button.	Loads the interactive 3D viewer.
Interacts with the model (rotate, zoom).	Responds to interactions in real-time.
Exits the 3D view.	Returns to normal item view.

Preconditions:

1. The item must have a 3D model available.
2. The user's device/browser must support 3D rendering.

Postconditions:

1. The user inspects the jewelry item in 3D.
2. Enhanced understanding of the item's physical appearance.

User Story:

As a user, I want to view jewelry items in 3D so that I can better understand their shape and design before making a purchase.

0.4 Use Case 4: Search Product

Name: Searching product

Identifier: UC4

Inputs:

1. Search keyword entered by user (e.g., "diamond ring", "ruby necklace").

Outputs:

1. List of jewelry products matching the search term.
2. AI-powered suggestions if available.

Basic Course:

Table 2.8: UC4_Search Product

Actor: Guest	System
Access the homepage or any public page.	Display the search bar (always accessible).
Enter a search keyword (e.g., "gold ring").	Capture the input and process the search query.
Submit the search request.	Retrieve relevant products from the database. Optionally display AI-enhanced recommendations based on keyword.
View list of matching products.	Render the results page with products, including filters (price, category, material, etc.).

Preconditions:

1. The system must have an active product catalog
2. User must have access to the search functionality (no login required).

Postconditions:

1. Matching products are displayed.
2. User can proceed to view product details, add to cart, or refine the search.

User Story:

As a user, I want to search for specific jewelry items so that I can quickly find products that match my preferences without browsing through all categories.

0.5 Use Case 5: Customize Product

Name: Customize Product

Identifier: UC5

Inputs:

1. Selection of jewelry type (e.g., ring, necklace, bracelet).
2. Choices for materials (gold, silver, platinum), gemstones (diamond, ruby, sapphire, etc.), engravings, size, style.

Outputs:

1. Real-time 3D preview of the customized jewelry.
2. Estimated price based on selected options.
3. Option to save design or proceed to checkout.

Basic Course:

Table 2.9: UC5_Custom

Actor: Guest	System
Navigate to the "Customize Jewelry" section.	Display customization interface and 3D viewer.
Select jewelry type (e.g., Ring).	Update customization options accordingly.
Choose materials, gemstones, engraving text, and size.	Apply selections and update 3D preview in real time.
(Optional) Request AI design suggestions.	Provide AI-driven suggestions based on trends and user preferences.
Confirm the customized design.	Calculate price dynamically based on choices.

Preconditions:

1. The system must have the customization module active
2. User must have access to the customization feature (login may be required to save designs).

Postconditions:

1. Customized jewelry is created.
2. The design is either saved to the user's profile or added to the shopping cart.

User Story:

As a user, I want to customize my jewelry by selecting stones, metals, styles, and engravings, so that I can create a personalized item that fits my tastes.

0.6 Use Case 6: View Product Details

Name: View Product Details

Identifier: UC6

Inputs:

1. User opens the website.
2. User navigates through the homepage, categories, or search results.
3. User clicks on a specific product to view its details.

Outputs:

1. Product detail page shows detailed product info.
2. Optional 3D preview and AI recommendations are displayed.

Basic Course:

Table 2.10: UC6_View Items

Actor: Guest	System
User accesses homepage or browses categories/search results.	System displays homepage or filtered product list.
User clicks on a product.	System displays the product detail page.
User scrolls to explore more.	System loads additional data (e.g., AI recommendations, 3D preview).

Preconditions:

1. Products must be available in the system's catalog.
2. The system must be accessible (no server downtime).

Postconditions:

1. The user successfully views detailed information about a product..
2. Viewed items may be recorded for personalization or future reminders.

User Story:

As a user, I want to view detailed information about a specific jewelry product so that I can evaluate its features and consider it before making a purchase.

0.7 Use Case 7: Forget Password

Name: Forget Password

Identifier: UC7

Inputs:

1. User clicks “Forgot Password” on login page.
2. Enters registered email address.

Outputs:

1. Confirmation email is sent.
2. User receives a reset password link.
3. User is redirected to reset password page.

Basic Course:

Table 2.11: UC7_Add to Reminder

Actor: Guest, Gmail	System
User selects "Forgot Password".	Displays input for email address.
User enters their registered email.	Sends password reset email via Gmail (<<extend>> Confirm Registration Email).
User clicks on email reset link.	Redirects to secure password reset form.
User enters and submits new password.	Updates user password in system.

Preconditions:

1. User has a registered account with a valid email.
2. Internet connection is active.

Postconditions:

1. User successfully resets password.
2. Email confirmation for password reset is recorded.

User Story:

As a customer, I want to reset my password via email so I can regain access to my account securely.

0.8 Use Case 8: Pay

Name: Pay

Identifier: UC8

Inputs:

1. Selected items
2. Payment details (credit card, e-wallet, etc.)

Outputs:

1. Payment confirmation.
2. Payment failure message (if any).

Basic Course:

Table 2.12: UC8_Pay

Actor: Customer	System
Select items and proceed to payment.	Display payment options.
Enter payment details.	Validate and process payment securely.
Confirm payment.	Confirm transaction, generate order ID, and notify success/failure.

Preconditions:

1. Customer has selected items to purchase.

Postconditions:

1. Payment is completed, order is generated.

User Story:

As a customer, I want to pay securely so that I can complete my jewelry purchase.

0.9 Use Case 9: Checkout

Name: Checkout

Identifier: UC9

Inputs:

1. Shipping information.
2. Billing address.
3. Order review.

Outputs:

1. Order summary and confirmation screen.

Basic Course:

Table 2.13: UC9_Checkout

Actor: Customer	System
After payment, review order details and confirm delivery address.	Display order summary and shipping details form.
Enter shipping address and contact info.	Validate inputs and prepare for final order confirmation.
Confirm checkout.	Update order with shipping information.

Preconditions:

1. Payment process has been initiated.

Postconditions:

1. Shipping details are stored with the order.

User Story:

As a customer, I want checkout so that my jewelry items are delivered to the right address.

0.10 Use Case 10: View Cart

Name: View Cart

Identifier: UC10

Inputs:

1. None (user action: view cart content).

Outputs:

1. List of items in the cart.
2. Option to proceed to checkout.

Basic Course:

Table 2.14: UC10_View Cart

Actor: Customer	System
Open "View Cart" page.	Display all products currently added to the cart with prices and quantities.
Review items.	Allow customer to edit quantities or remove products before finalizing checkout.

Preconditions:

1. Customer has items in cart.

Postconditions:

1. Cart content is ready for checkout.

User Story:

As a customer, I want to view my cart to verify the products and make changes before checking out.

0.11 Use Case 11: View Orders

Name: View Orders

Identifier: UC11

Inputs:

1. Order ID (typically retrieved from order history or confirmation email).

Outputs:

1. Order details (items, shipping status, payment status, delivery date).
2. Order status (e.g., pending, shipped, delivered).

Basic Course:

Table 2.15: UC11 _ View Orders

Actor: Customer	System
Log into the system and navigate to "Order History" or "My Orders".	Display list of past orders, including order IDs and statuses.
Select an order to view detailed information.	Retrieve and display order details, including: - Product details (name, quantity, price). - Shipping and billing addresses. - Payment status. - Delivery date and status.
Review order details.	Provide options to track shipment (if applicable).

Preconditions:

1. Customer has made at least one order.
2. Customer is logged into the system

Postconditions:

1. Customer has reviewed order details.

User Story:

As a customer, I want to view my past orders so that I can track their status and verify the details.

0.12 Use Case 12: Leave Feedback/Review

Name: Leave Feedback/Review

Identifier: UC12

Inputs:

1. Rating (e.g., 1 to 5 stars).
2. Written review (optional).

Outputs:

1. Confirmation message (Review successfully submitted).
2. Error message (if review submission fails).

Basic Course:

Table 2.16: UC12_Leave Feedback/Review

Actor: Customer	System
Navigate to the order history or product page.	Display list of purchased items with option to leave feedback/review.
Select an item to review.	Display review form with options to rate and write a comment.
Rate the item and optionally write a review.	Validate review input (rating, text).
Submit the review.	Store the review in the system and update the product's average rating.
Confirmation message is shown (or error message if validation fails).	Provide confirmation of successful submission.

Preconditions:

1. Customer has purchased the item they wish to review
2. Customer is logged into the system

Postconditions:

1. The review is successfully submitted and saved.
2. The product's average rating is updated.

User Story:

As a customer, I want to leave feedback on products I have purchased so that I can share my experience with other users.

0.13 Use Case 13: Add to Cart

Name: Add to Cart

Identifier: UC13

Inputs:

1. Product selection (jewelry item).
2. Quantity (if applicable).

Outputs:

1. Confirmation message (Item added to cart).
2. Updated cart view (if applicable).

Basic Course:

Table 2.17: UC13 _Add to Cart

Actor: Customer	System
Browse through the product catalog.	Display available products with details (e.g., name, price, description).
Select a product to purchase.	Display product details (e.g., options for size, material, quantity).
Choose quantity and click "Add to Cart".	Validate the selection (e.g., quantity, available stock).
Confirm the item is added to the cart.	Update the cart with the selected item, show updated cart content.
Show updated cart count and total.	Display a confirmation message and updated cart view.

Preconditions:

1. Customer is browsing the catalog.

Postconditions:

1. The selected item is added to the cart.
2. The cart is updated with the new item.

User Story:

As a customer, I want to add items to my cart so that I can proceed to checkout when I'm ready to purchase.

0.14 Use Case 14: Confirm Registration Email

Name: Confirm Registration Email

Identifier: UC14

Inputs:

1. Registered email address of the user.
2. Verification link/token.

Outputs:

1. Confirmation that the email address has been verified.
2. Error message (if token is invalid or expired).

Basic Course:

Table 2.18: UC14_ Confirm Registration Email

Actor: Gmail	System
Deliver registration confirmation email to the user.	Include verification link/token in the email body.
User clicks the link from Gmail.	Receive and validate the token.
	Mark user account as verified.
	Redirect the user to a confirmation page (e.g., "Email successfully verified").

Preconditions:

1. The user has completed the sign-up form with a valid email.
2. The system has generated and sent a verification token to that email.

Postconditions:

1. The user's email is marked as verified.
2. The user can now log in and access full system features.

User Story:

As a system, I want to confirm the user's email via Gmail so that only valid users can activate their accounts and enhance security.

0.15 Use Case 15: Suggest Personalized Items

Name: Suggest Personalized Items

Identifier: UC15

Inputs:

1. Chatbot detects user activity (e.g., item views, search queries).
2. User interacts with the chatbot

Outputs:

1. Display a list of AI-personalized item suggestions in the chat interface.
2. Option to view, add to cart, or dismiss suggestions.

Basic Course:

Table 2.19: UC15_ Confirm Registration Email

Actor: Chatbot	System
Analyze user behavior and preferences.	Query AI recommendation engine.
Suggest personalized items in chat.	Display interactive item cards in chat.
(Optional) Receive feedback from user (e.g., "Not interested").	Adjust future suggestions accordingly

Preconditions:

1. The user has interacted with the system (e.g., browsed items).
2. The chatbot is active and integrated with recommendation engine.

Postconditions:

1. User receives AI-based personalized suggestions.
2. Suggestions may influence user actions (e.g., add to cart).

User Story:

As a chatbot, I want to suggest personalized jewelry items to the user based on their preferences so they can find what suits them best more easily.

0.16 Use Case 16: Manage Inventory

Name: Manage Inventory (The function of Admin role is not complete, so the Actor will change to Monitor in this Use case Diagram)

Identifier: UC16

Inputs:

1. Monitor logs into the system.
2. Monitor selects "Inventory Management" from the dashboard.
3. Monitor adds, updates, or removes product entries.

Outputs:

1. Confirmation of successful inventory changes.
2. Updated product list is reflected across user-facing systems.

Basic Course:

Table 2.20: UC16_ Answer Product's Related Question

Actor: Monitor	System
Login to admin panel.	Authenticate admin credentials.
Navigate to inventory section.	Display inventory dashboard.
Add, edit, or remove item details.	Update product database accordingly.
Save changes.	Show success notification and update frontend listings.

Preconditions:

1. Monitor must be authenticated.
2. System must allow access to inventory database.

Postconditions:

1. Product list is updated in real-time.
2. Changes are visible to all users browsing the platform.

User Story:

As an monitor, I want to manage the product inventory so that the system always reflects accurate and up-to-date product availability and details.

3.4. System Design

3.4.1. System Architecture

Model–view–controller (MVC): a software design pattern commonly used for developing user interfaces that divides the related program logic into three interconnected elements. These elements are the internal representations of information (the model), the interface (the view) that presents information to and accepts it from the user, and the controller software linking the two.

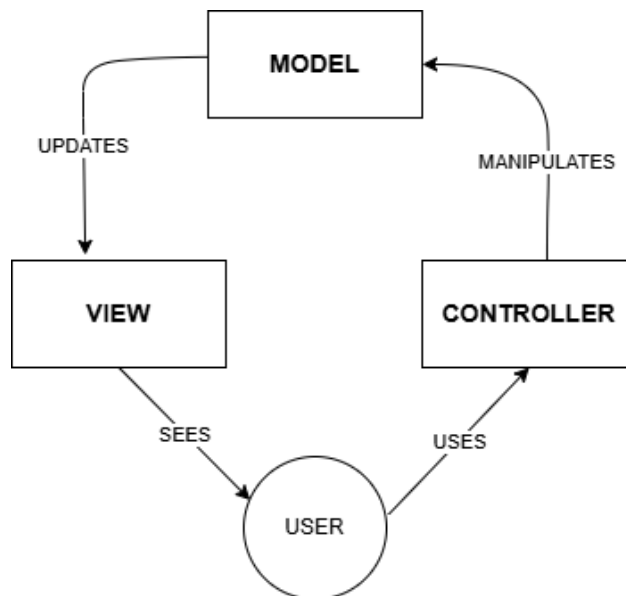


Figure 2.2: MVC

REST (representational state transfer or RESTful API): a software architectural style designed for the World Wide Web, emphasizing uniform interfaces, independent component deployment, scalable interactions, layered architecture for caching, security enforcement, and legacy system encapsulation.

3.4.2. Entity Relationship Diagram

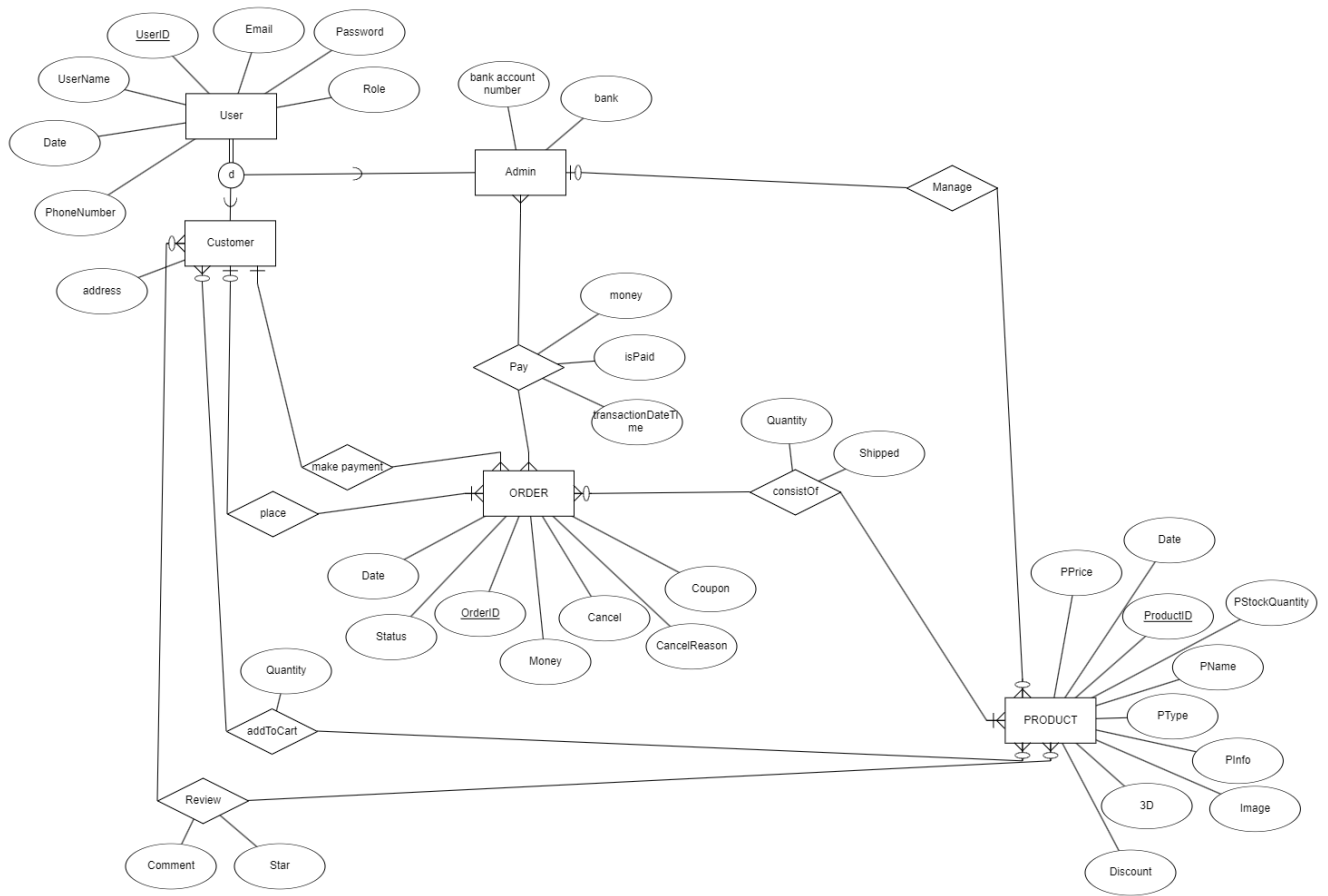


Figure 2.3: Entity Relationship Diagram

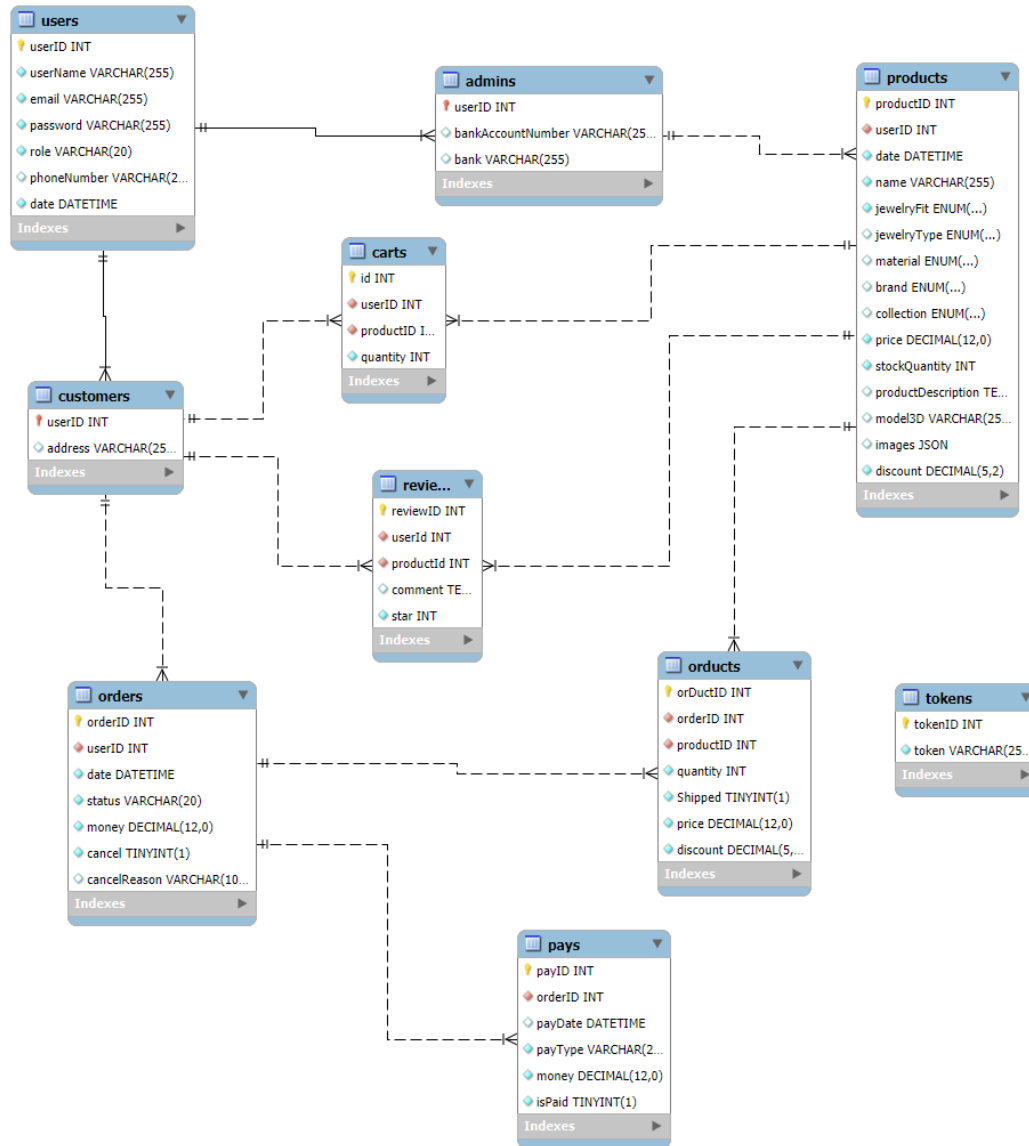


Figure 2.4: Table Schema

Schema description:

Table 2.21: Schema description

No.	Entity	Description
1	users	General users of the system, including both admins and customers.
2	admins	Users with administrative privileges who manage the system.
3	customers	Users who browse, order, purchase, etc products from the platform.
4	products	Items or goods available for sale on the platform.
5	carts	Virtual baskets where customers add products before purchasing.

6	reviews	Feedback or ratings provided by customers for purchased products
7	orders	Transactions made by customers to buy products.
8	orducts	(Likely a typo of "OrderProducts") Links specific products to an order.
9	pays	Payment information related to customer orders.
10	tokens	This table stores tokens that are allowed for one-time use only. It is primarily used for password reset functionality but can also store other one-time-use tokens for different purposes as the website expands.

3.4.3. Class Diagram (High-Level)

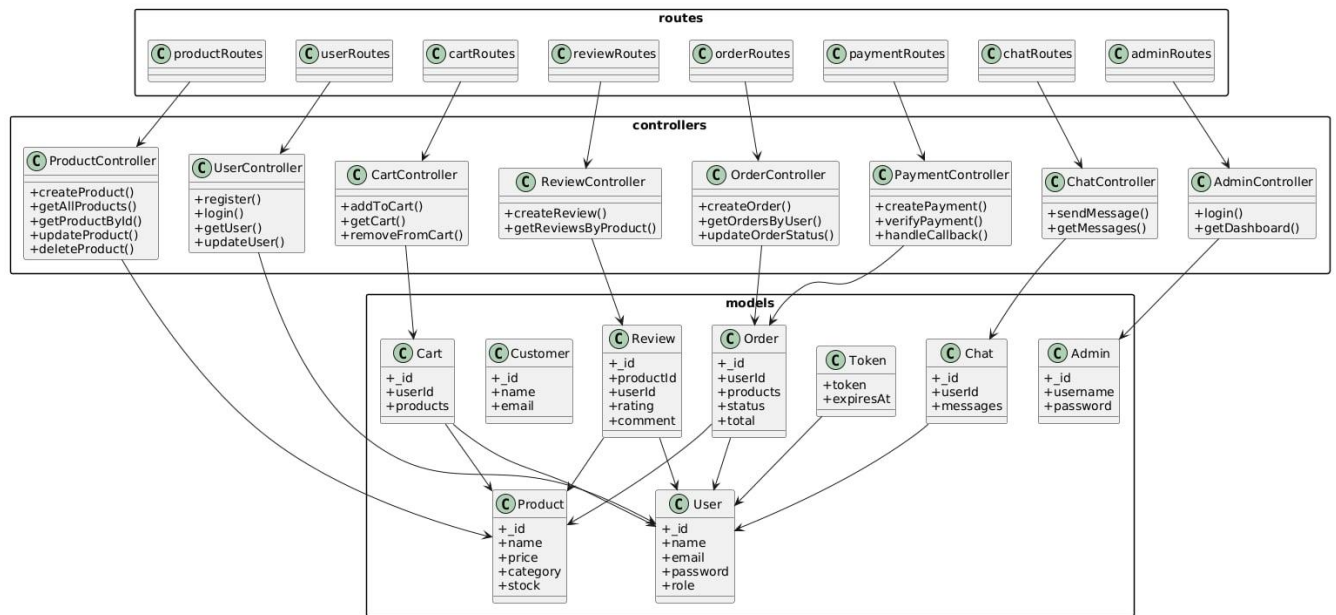


Figure 2.5: Class Diagram

Explanation:

The Class Diagram for the AI Shopping System defines the static structure, detailing classes, their attributes, methods, and relationships to support the updated use case diagram. The diagram includes the following key components:

- **Routes:** Serve as the entry points, including `productRoutes`, `userRoutes`, `cartRoutes`, `reviewRoutes`, `orderRoutes`, `paymentRoutes`, `chatRoutes`, and `adminRoutes`, mapping to respective controllers.

- **Controllers:** Handle business logic, including ProductController (+createProduct(), +getAllProducts(), +getProductById(), +updateProduct(), +deleteProduct()), UserController (+register(), +login(), +getUser(), +updateUser()), CartController (+addToCart(), +removeFromCart()), ReviewController (+createReview(), +getReviewsByProduct()), OrderController (+createOrder(), +getOrdersByUser(), +updateOrderStatus()), PaymentController (+createPayment(), +verifyPayment(), +handleCallback()), ChatController (+sendMessage(), +getMessages()), and AdminController (+login(), +getDashboard()).
- **Models:** Represent data entities, including Product (+id, +name, +price, +category, +stock), User (+id, +name, +email, +password, +role), Cart (+id, +userId, +products), Customer (+id, +name, +email), Review (+id, +productId, +userId, +rating, +comment), Order (+id, +productId, +userId, +status, +total), Token (+token, +expiresAt), Chat (+id, +userId, +messages), and Admin (+id, +name, +password).

Key Relationships:

- Product is managed by ProductController, linked to Product model, supporting "View Product Details" and "Customize Product."
- User and Customer are handled by UserController, linked to User and Customer models, supporting "Sign Up," "Login," and "ForgotPassword."
- Cart is managed by CartController, linked to Cart model, supporting "View Cart" and "Add to Cart."
- Order is processed by OrderController, linked to Order model, supporting "View Orders" and "Checkout."
- Payment is handled by PaymentController, linked to external systems and Token model, supporting "Pay."

- Chat is managed by ChatController, linked to Chat model, supporting customer assistance via Chatbot.
- Admin is managed by AdminController, linked to Admin model, supporting "Manage Inventory" and trend analysis for "Suggest Personalized Products."

3.4.4. Sequence Diagram

3.4.4.1.Suggest Personal

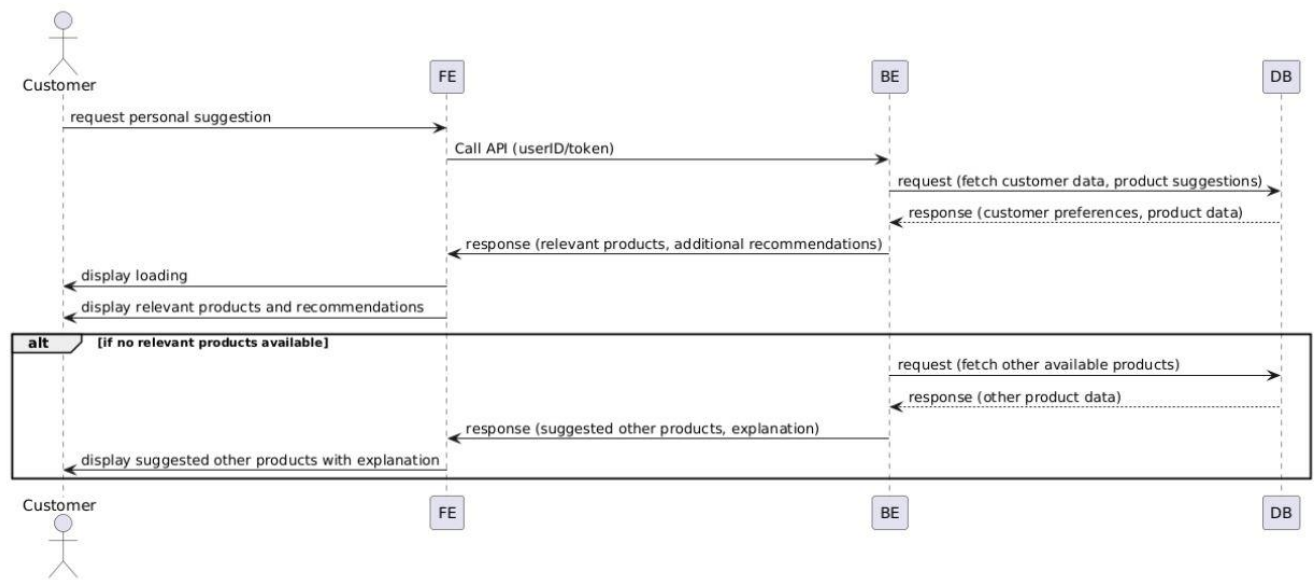


Figure 2.6: Sequence Diagram_Suggest Personal

This sequence diagram illustrates the interaction between a Customer, the Frontend (FE), the Backend (BE), and the Database (DB) when a customer requests personalized product suggestions.

Main Flow:

- Customer → FE: The customer initiates the process by entering a search query (e.g., "gold ring") to request product suggestions.
- FE → BE: The frontend sends the search request to the backend via an API call, including the userID and search query.

- BE → DB: The backend queries the database to fetch available product data matching the search query.
- DB → BE: The database returns the matching product data to the backend.
- BE → FE: The backend processes the data to generate personalized suggestions based on the customer's preferences and search history, then sends the combined search results and suggestions to the frontend.
- FE → Customer: The frontend displays a loading screen while processing the data
- FE → Customer: The frontend renders the search results and personalized suggestions for the customer to view.

Alternative Flow (alt – if no suggestions are available):

- If the backend cannot generate suitable suggestions:
 - BE → FE: The backend sends an error message along with the search results.
 - FE → Customer: The frontend displays the search results with a message indicating no personalized suggestions are available

3.4.4.2.Payment Process

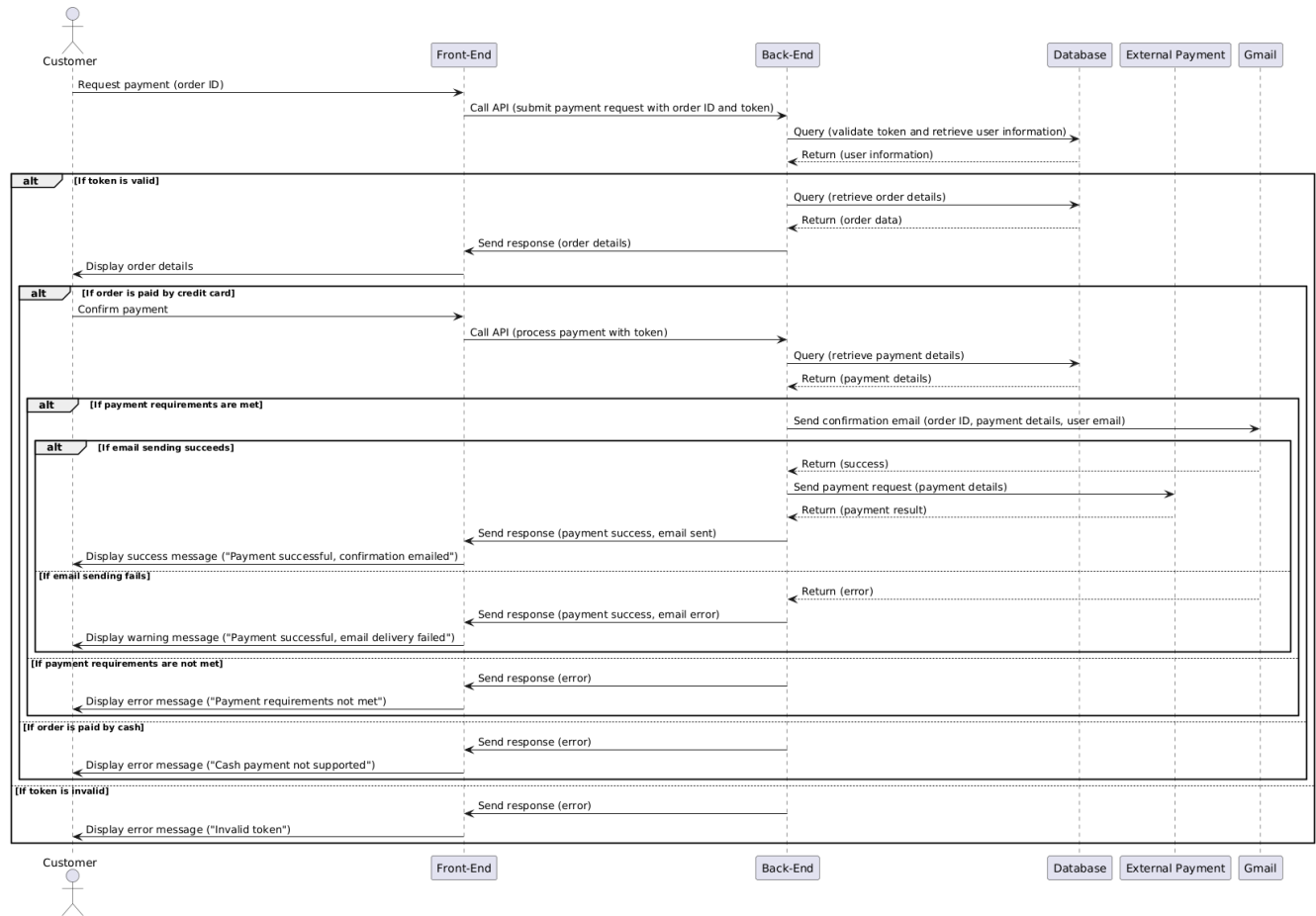


Figure 2.7: Sequence Diagram _Payment Process

This diagram describes the sequence of interactions between the Customer, Front-End (FE), Back-End (BE), Database (DB), and External Payment Service (EP) when a customer attempts to make a payment for an order.

Main Flow:

- Customer → FE: The customer sends a request to make a payment, providing the order ID
- FE → BE: The frontend sends an API call to the backend, including the order ID and a token for authentication.
- BE → DB: The backend queries the database to validate the token and retrieve associated user information.
- DB → BE: The database returns the user information.

Alternative Flow – Valid Token:

- BE → DB: If the token is valid, the backend queries the database to get order details.
- DB → BE: The order data is returned to the backend.
- BE → FE: The backend responds to the frontend with the order information.
- FE → Customer: The frontend displays the order details to the customer.

Sub-flow – Credit Card Payment:

- Customer → FE: The customer confirms the payment.
- FE → BE: The frontend calls the backend to process the payment using the token.
- BE → DB: The backend retrieves payment details from the database.
- DB → BE: The payment details are returned.
- BE → EP: If payment requirements are satisfied, the backend sends a payment request to the external payment system.
- EP → BE: The external payment service returns the payment result.
- Back-End → Gmail: The backend sends a confirmation email with the order ID, payment details, and user email.
- Gmail → Back-End: The email service returns the send success or error result.
- BE → FE: The backend informs the frontend of a successful payment.
- FE → Customer: The frontend displays a success message to the customer.

Sub-flow – Payment Requirements Not Met:

- If the payment requirements are not met:
 - **BE → FE:** Sends an error response.
 - **FE → Customer:** Displays the message "Payment requirements not met".

Alternative – Cash Payment Not Supported:

- If the customer selects **cash payment**:

- **BE → FE:** Sends an error response.
- **FE → Customer:** Displays "Cash payment not supported".

Alternative Flow – Invalid Token:

- If the token is invalid:
 - **BE → FE:** Sends an error response.
 - **FE → Customer:** Displays the message "Token required".

3.4.4.3. Leave Feedback

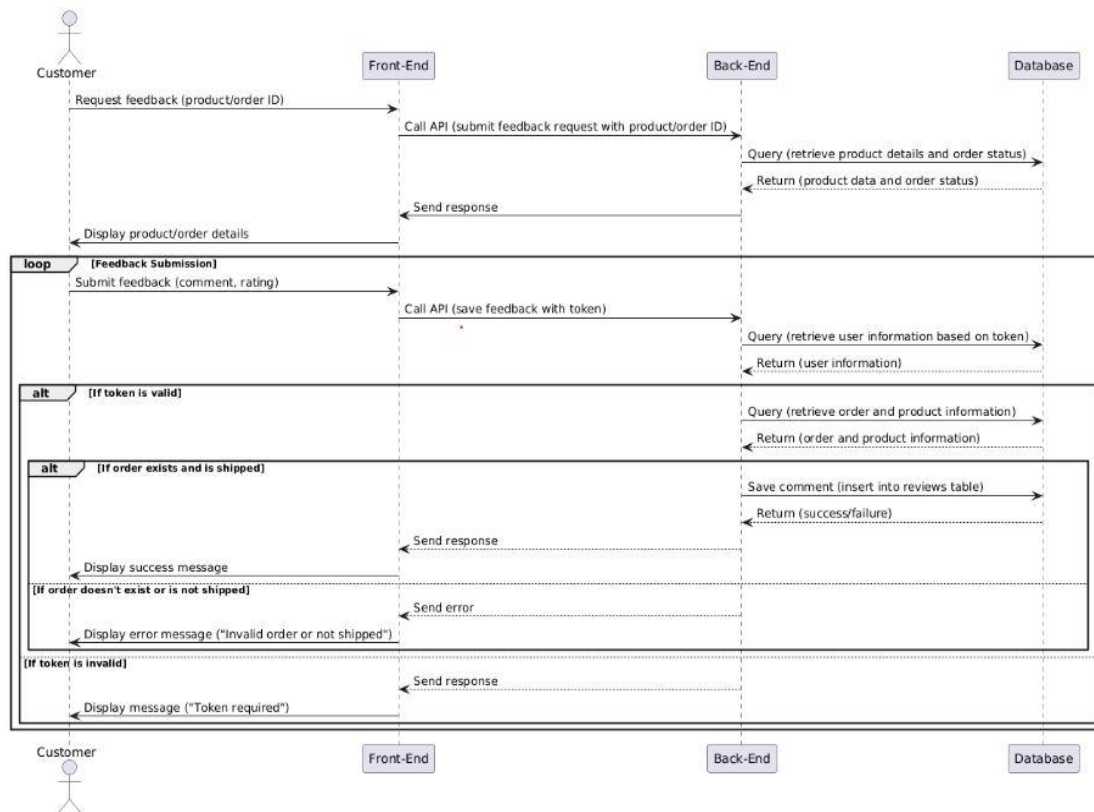


Figure 2.8: Sequence Diagram _Leave feedback

Requesting Feedback Details

- Customer → Front-End (FE): The customer initiates the process by requesting to give feedback, providing either a product ID or an order ID.

- FE → Back-End (BE): The frontend calls the backend API, submitting the request with the provided product or order ID.
- BE → Database (DB): The backend queries the database to retrieve product details and the status of the order.
- DB → BE → FE → Customer: The information is returned and displayed to the customer.

Feedback Submission Loop: This part represents a loop that continues until the feedback is successfully submitted or fails.

- Customer → FE: The customer fills in and submits feedback (comment and rating).
- FE → BE: The frontend sends the feedback data to the backend along with a token to verify the customer.
- BE → DB: The backend queries the database using the token to get user information.
- DB → BE: Returns user data.

A. Token Validation Success

- BE → DB: The backend uses the user information to fetch the associated order and product data.
- DB → BE: Returns the order and product details.
- If the order exists and is shipped:
 - BE → DB: The backend saves the comment by inserting it into the reviews table.
 - DB → BE → FE → Customer: Response is sent back to the frontend, and the customer is shown a success message.
- Else (if the order doesn't exist or is not shipped):
 - BE → FE → Customer: An error response is sent back and the message "Invalid order or not shipped" is shown to the customer.

B. Token Validation Failure

- BE → FE → Customer: If the token is invalid, the customer receives a message indicating that

authentication is required ("Token required").

3.4.4.4.3D Product View Request

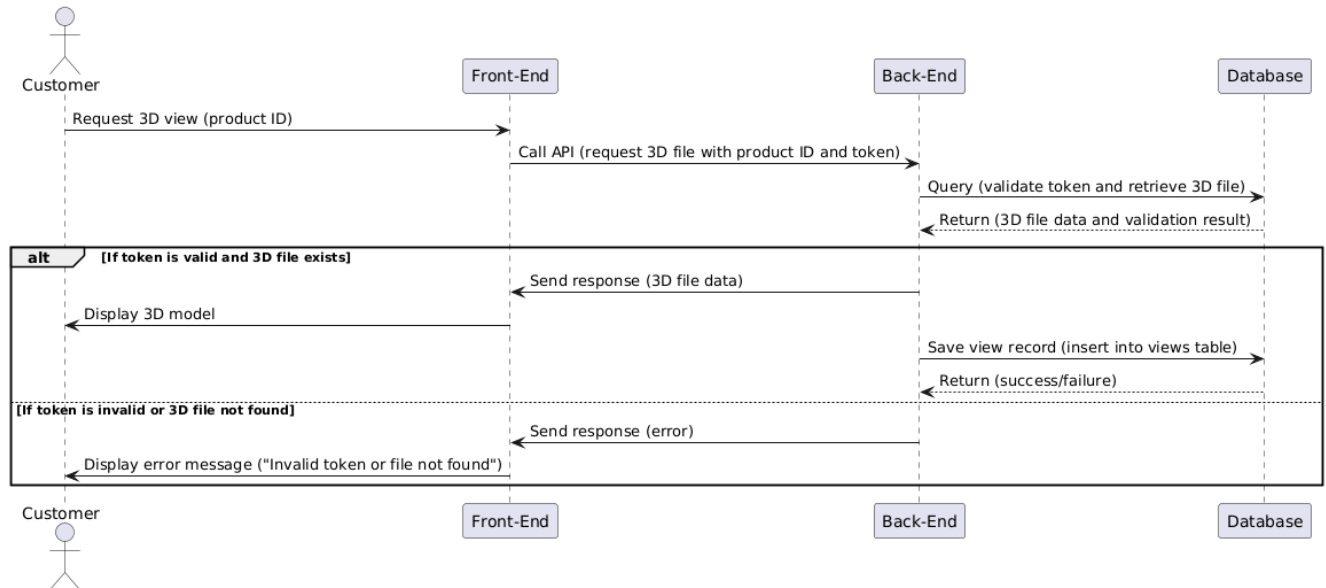


Figure 2.9: Sequence Diagram _ 3D Product View Request

This sequence diagram describes how a Customer requests to view a 3D model of a product. It details the validation, data retrieval, and logging process involving the Front-End (FE), Back-End (BE), and Database (DB).

Main Flow:

- Customer → FE: The customer initiates the process by requesting a 3D view for a product using the product ID.
- FE → BE: The frontend sends an API call to the backend, including the product ID and token.
- BE → DB: The backend queries the database to validate the token and retrieve the corresponding 3D file.
- DB → BE: The database returns the 3D file data along with the result of the token validation.
- BE → DB: If the token is valid and the 3D file exists, the backend logs this action by inserting a view record into the views table.
- DB → BE: The database confirms the success or failure of the insert operation

- BE → FE: The backend sends a response to the frontend containing the 3D file data.
- FE → Customer: The frontend displays the 3D model to the customer.

Alternative Flow (alt – if token is invalid or 3D file not found):

If the token is invalid or the 3D file does not exist:

- BE → FE: The backend sends an error response to the frontend.
- FE → Customer: The frontend displays an error message indicating “Invalid token or file not found”.

3.4.5. User Interface Design



Figure 2.10: Default Header



Figure 2.11: Logged in Header

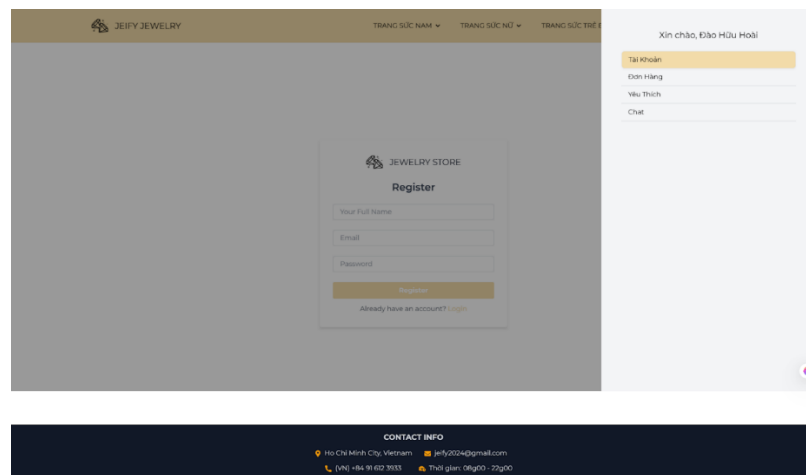


Figure 2.12: Sub navigation bar

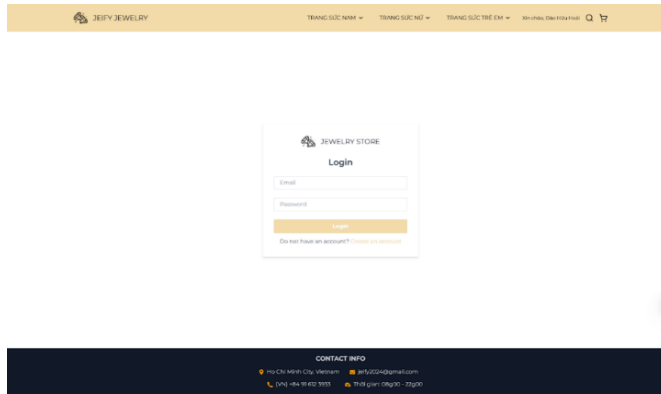


Figure 2.13: Log in page

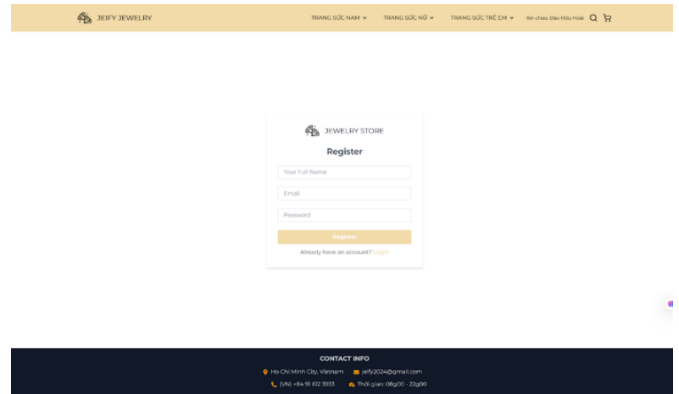


Figure 2.14: Register info

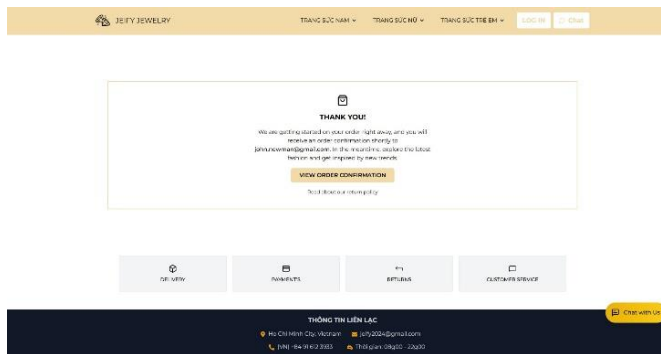


Figure 2.15: Confirm email

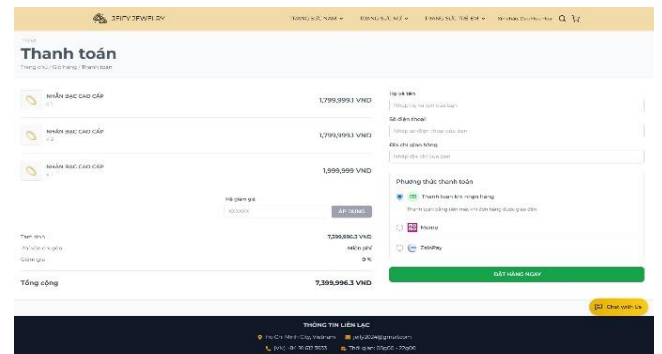


Figure 2.16: Checkout

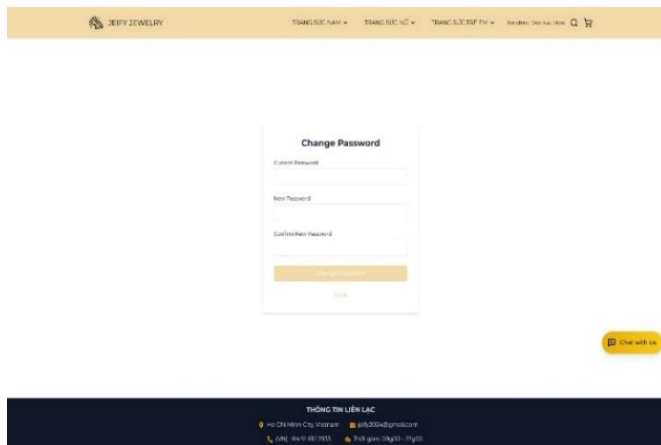


Figure 2.17: Change password

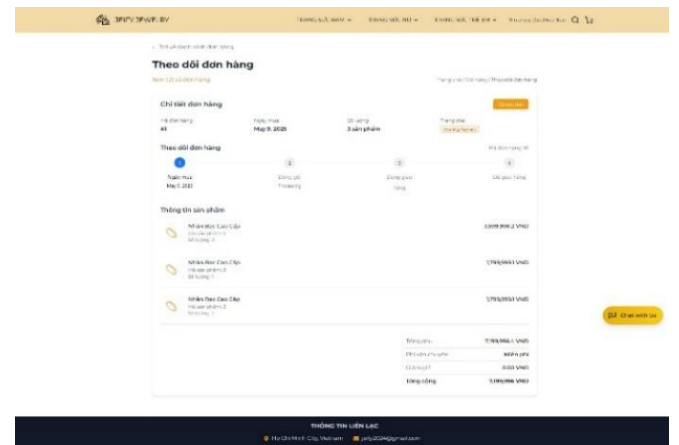


Figure 2.18: Order

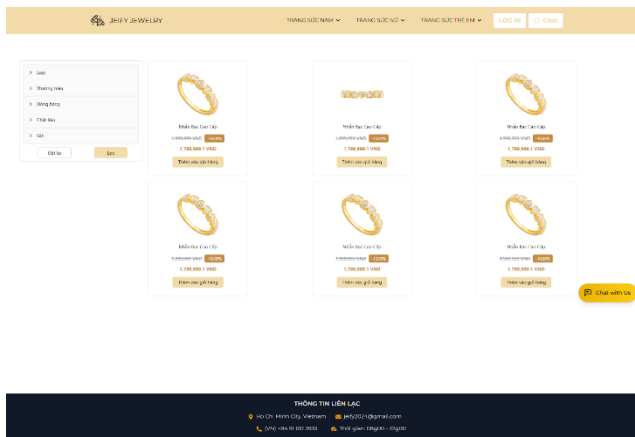


Figure 2.19: List product

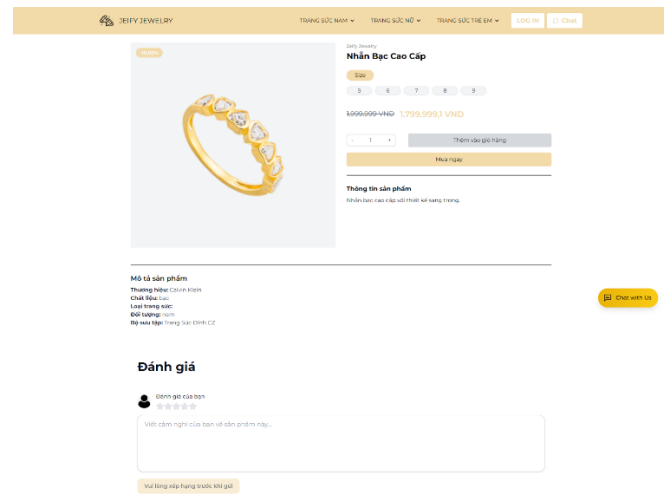


Figure 2.20: Product detail page

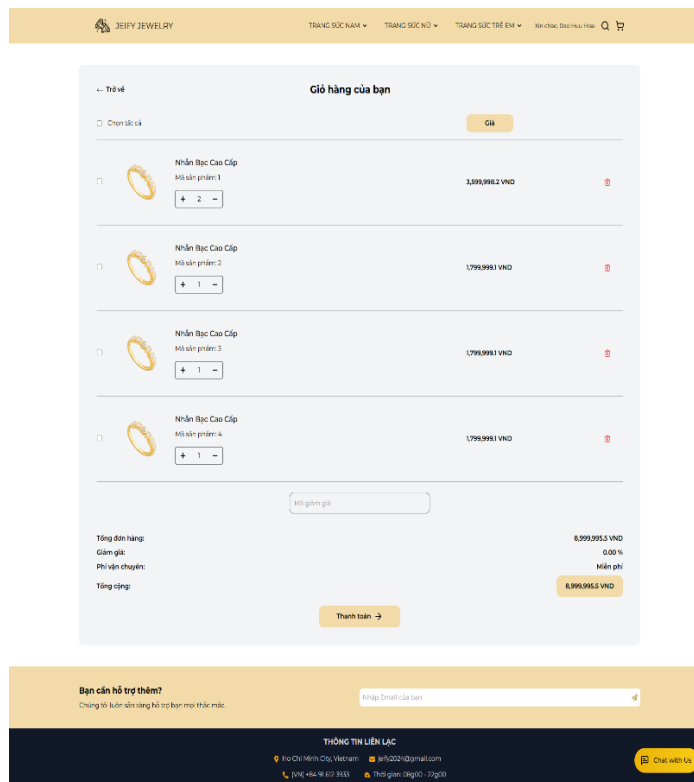


Figure 2.21: Cart page

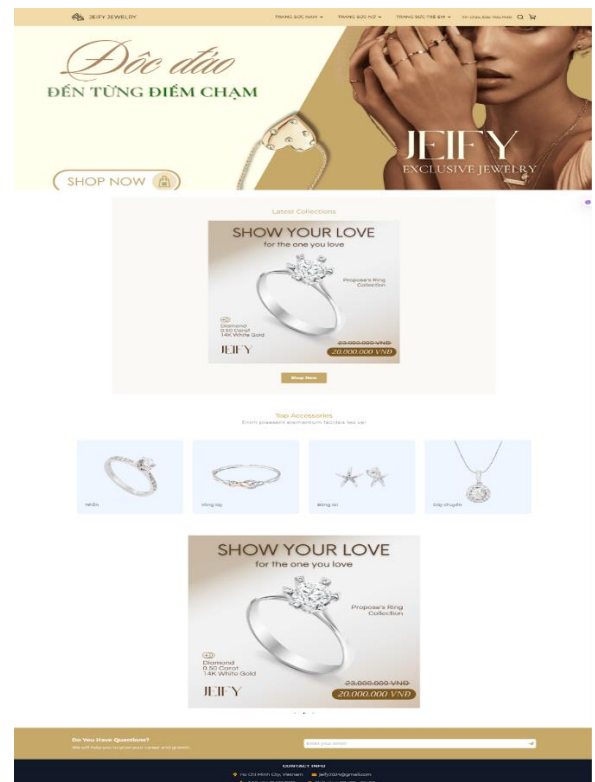


Figure 2.22: Home page

CHAPTER 4

IMPLEMENT AND RESULT

4.1. Implement

4.1.1. Front-end

4.1.1.1. Language

The frontend of the AI Shopping System was developed using:

- **HTML:** Used for structuring the layout of the web pages.
- **CSS:** Utilized to enhance the visual presentation and ensure responsive design across various devices.
- **JavaScript:** Employed for handling client-side interactivity, dynamic rendering, and logic within the application.

4.1.1.2. Framework

React.js was utilized as the primary framework for frontend development in the AI Shopping System, with the following key advantages:

- **Component-Based Architecture:** Facilitated the creation of modular and reusable user interface components, enhancing the system's maintainability and scalability.
- **Development Efficiency:** Improved the overall efficiency of the development process by promoting code reusability and clarity.
- **Single-Page Application Navigation:** Integrated React Router to enable smooth navigation between different views without requiring full-page reloads, thereby enhancing the fluidity and responsiveness of the user experience.

4.1.2. Back-end

4.1.2.1.Environment

Node.js (JS) was chosen as the backend environment due to the following reasons:

- **Cross-Platform Compatibility:** Just like React Native allows code sharing between iOS and Android, Node.js runs on various operating systems like Windows, macOS, and Linux. This simplifies backend development and deployment regardless of the server environment.
- **Real-time Features:** Node.js is event-driven and excels at handling real-time applications. This is beneficial for features like live chat functionalities or updating inventory stock levels.

4.1.2.2.Framework

Express.js was chosen as the development framework for the benefits:

- **Fast and Minimalist:** Express.js is a lightweight framework on top of Node.js that allows for rapid development of APIs. This is ideal for building a scalable backend for the gift shop app.
- **Flexibility:** Express offers a great deal of flexibility in building APIs. You can customize it to fit the specific needs of your gift shop app, like handling product data, user accounts, and order processing.

4.1.2.3.Database

MySQL was selected as the database solution for the following reasons:

- **Relational Data Structure:** MySQL is well-suited for structured data storage, such as user accounts, product details, orders, and payment history. The relational model ensures data consistency through normalization and supports complex queries using SQL.

- **Performance and Reliability:** MySQL offers robust performance and stability, even under high load conditions. It is widely used in production systems, ensuring reliable handling of transactions and secure data storage.

4.2. Results of System's Architecture

4.2.1. Backend Overview

The backend architecture of the AI Shopping System was designed using a layered and modular structure to ensure maintainability, scalability, and efficiency. It was developed using **Node.js** as the runtime environment and **Express.js** as the application framework, providing a lightweight, high-performance base for building RESTful APIs. Key functionalities and architectural features of the backend include:

- **API Routing Layer:** Express.js was used to define modular routes for handling various operations such as user authentication, product management, order processing, and admin controls. Each route is organized based on its domain responsibility to improve readability and separation of concerns.
- **Controller and Service Layers:** Core logic was abstracted into controller and service layers. Controllers receive and validate HTTP requests, while services handle business logic such as inventory management, discount calculation, and user role validation.
- **Real-time and Asynchronous Handling:** Leveraging Node.js's non-blocking I/O model, the system can efficiently handle concurrent user requests and real-time functionalities, such as stock level updates and admin notifications.
- **Security and Middleware:** Middleware functions were implemented to manage session control, user authentication, input validation, and error handling. This includes token-based authentication for securing APIs and protecting sensitive endpoints.

4.2.2. Frontend Overview

The frontend architecture was structured around a modular, component-driven design using ReactJS. Major UI components developed include:

- **Default Header and Logged-in Header:** Navigation bars dynamically updated based on user authentication status.
- **Sub Navigation Bar:** A secondary menu providing quick filters and product search options.
- **Home Page:** A promotional landing page showcasing featured jewelry items and latest offers.
- **Login and Registration Pages:** Secure forms allowing new users to sign up and existing users to log in.
- **Product Listing and Filter Page:** Displays products along with sorting and filtering capabilities to enhance search precision.
- **Product Detail Page:** Provides detailed views, including 3D real-time jewelry previews and customization options.
- **Cart Page:** Allows users to review selected items, adjust quantities, and proceed to checkout.
- **Admin Dashboards:** Separate admin views for managing customer orders and employee data.

All components were designed with responsiveness, performance optimization, and user experience (UX) best practices in mind, ensuring a consistent look and feel across different devices and screen sizes.

CHAPTER 5

DISCUSSION, TESTING AND EVALUATION

5.1. Discussion

Before initiating the testing and evaluation phase, we conducted an in-depth discussion to clearly define the objectives, scope, functional requirements, and priorities of the AI Jewelry Shopping System. This process ensured that all testing activities were strategically aligned with the system's goal of delivering a personalized, intelligent, and seamless jewelry shopping experience

5.2. Testing methodologies employed

5.2.1. Unit tests

Table 3.1: Unit test applications

Feature	Component	Description
Jewelry Customization	3D Preview Rendering	Check if the 3D preview updates correctly when the user selects different designs.
User Authentication	Login & Signup	Verify credential validation and account creation using valid/invalid inputs.
	Session Handling	Test if JWT tokens are generated, stored, and validated properly.
Payment Integration	Payment Gateway	Check integration with payment providers using mock responses.
	Order Confirmation	Ensure users receive correct confirmation details and invoice generation.

5.2.2. Integration tests

Table 3.2: Integration test applications

Feature	Description
AI Recommendation and Product Display	Ensure that the AI-based recommendation engine correctly fetches and displays personalized jewelry products on the main shopping interface.
Cart and Pricing Engine	Check that when products are added or removed from the cart, the pricing engine updates the total cost correctly. Validate the link between cart and pricing modules.
Payment and Order Confirmation	Ensure that successful payments trigger order confirmations via email or in-app messages. Validate the interaction between the payment gateway and notification system.

5.2.3. System tests

Table 3.3: System test applications

Feature	Description
Functional Testing (End-to-End Shopping Process)	Simulate a complete shopping journey: user logs in, receives AI-powered product suggestions, uses filters/voice search, adds jewelry to cart, completes payment, and receives confirmation. Ensure all steps function smoothly together.
Performance Testing	Test the performance of the system under various load conditions (e.g., multiple users browsing, searching, and purchasing simultaneously). Ensure the system remains responsive and stable under high traffic.
Security Testing	Conduct tests to protect user data, including checking for vulnerabilities like SQL injection, cross-site scripting (XSS), and insecure storage. Ensure secure payment processing and encrypted data transmission.

5.3. Test Cases

5.3.1. Front-end

Automation Testing Test Cases

Table 3.4: Test Case 1 Information

Test Case 1: User Authentication				
System: AI Shopping System			Subsystem: User Services	
Designed by: Mr. Lê Thiên Doanh			Design Date: 09/04/2025	
Executed by: Mr. Nguyễn Thành Nam			Execution Date:03/05/2025	
Short Description: Test the login functionality using valid and invalid credentials.				
System Testing: Functional Testing				
Pre-conditions: - User accounts already registered in the database. - Login page is accessible via URL			Post-conditions: - Successful login redirects users to the homepage. - Unsuccessful login shows an appropriate error message.	
Description				
Step	Action	Expected System Response	Pass/Fail	Comments
1	Access login page	Page displays login form	Pass	Page loads properly
2	Submit valid credentials	Redirect to homepage	Pass	Login successful.
3	Submit invalid credentials	Error: "Invalid credentials"	Pass	Proper validation

Table 3.5: Test Case 2 Information

Test Case 2: Add Product to Cart	
System: AI Shopping System	Subsystem: Cart Services
Designed by: Mr.Đào Hữu Hoài	Design Date: 04/04/2025
Executed by: Mr.Hồ Thiên Trường	Execution Date: 03/05/2025
Short Description: Test adding products to the shopping cart from the product list.	
System Testing: Functional Testing	
Pre-conditions: - User is logged into the system. - Product catalog is available and displayed.	Post-conditions: Selected products appear in the cart with correct quantity and price.

<u>Description</u>				
Step	Action	Expected System Response	Pass/Fail	Comments
1	Open product listing page.	List of products displayed properly.	Pass	Product list loaded successfully.
2	Click "Add to Cart" on a product.	Product added to cart, cart icon updates.	Pass	Product added successfully.
3	Open cart page.	Cart shows correct product information.	Pass	Cart displays correct product details.

Table 3.6: Test Case 3 Information

Test Case 3: Checkout Process				
System: AI Shopping System			Subsystem: Payment and Checkout	
Designed by: Mr.Đào Hữu Hoài			Design Date: 04/04/2025	
Executed by: Mr.Hồ Thiên Trường			Execution Date:03/05/2025	
Short Description: Test the full checkout and payment process.				
System Testing: Functional Testing				
Pre-conditions: - User has added at least one product to cart. - User is logged in.			Post-conditions: - Order is placed successfully. - Confirmation email or message is sent to the user.	
Description				
Step	Action	Expected System Response	Pass/Fail	Comments
1	Open cart page.	Cart displays selected products correctly.	Pass	Cart loaded successfully.
2	Click "Proceed to Checkout".	Checkout form appears for shipping and payment details.	Pass	Checkout form displayed successfully.
3	Fill in payment and shipping information.	System accepts valid input.	Pass	Payment and shipping info accepted.
4	Confirm and place order.	Order placed and confirmation displayed.	Pass	Order placed successfully.

Manual Testing Test Cases

Table 3.7: Test Case 4 Information

Test Case 4: 3D Jewelry Preview Interaction				
System: AI Shopping System		Subsystem: Jewelry Customization		
Designed by: Mr. Huỳnh Hữu Nghĩa		Design Date: 26/04/2025		
Executed by: Mr. Nguyễn Thành Nam		Execution Date: 03/05/2025		
Short Description: Test smoothness and responsiveness of the 3D jewelry preview.				
System Testing: Functional Testing				
Pre-conditions: - Product detail page includes a 3D preview feature. - Internet connection is stable for rendering 3D models.		Post-conditions: 3D model responds smoothly to rotate and zoom. - Updated customizations reflect in real-time.		
Description				
Step	Action	Expected System Response	Pass/Fail	Comments
1	Open product detail page.	3D jewelry model loads automatically.	Pass	3D preview displayed successfully.
2	Rotate and zoom the 3D model.	Model rotates and zooms smoothly.	Pass	Smooth interaction without lag.
3	Change gemstone or metal type.	3D model updates appearance instantly.	Pass	Appearance updated successfully.

Table 3.8: Test Case 5 Information

Test Case 5: User Interface Responsiveness Check	
System: AI Shopping System	Subsystem: Frontend UI
Designed by: Mr. Nguyễn Hoàng Bảo Khang	Design Date: 20/04/2025
Executed by: Mr. Nguyễn Trí Dũng	Execution Date: 03/05/2025
Short Description: Test UI layout and responsiveness on multiple devices.	
System Testing: Functional Testing	
Pre-conditions: Website is deployed.	Post-conditions: UI adapts correctly to different screen sizes without layout breaking.

• Testing devices (Desktop, Tablet, Mobile) available.				
<u>Description</u>				
Step	Action	Expected System Response	Pass/Fail	Comments
1	Open website on desktop.	Desktop view with full layout displayed.	Pass	Layout displayed properly on desktop.
2	Open website on tablet.	Tablet-optimized layout loaded correctly.	Pass	Layout responsive on tablet.
3	Open website on mobile.	Mobile-friendly view displayed.	Pass	Mobile view rendered correctly.

Table 3.9: Test Case 6 Information

Test Case 6: Product Information Accuracy in Cart				
System: AI Shopping System		Subsystem: Cart Services		
Designed by: Mr.Đào Hữu Hoài		Design Date: 28/04/2025		
Executed by: Mr.Hồ Thiên Trường		Execution Date: 03/05/2025		
Short Description: Verify the correctness of product details in the cart				
System Testing: Functional Testing				
Pre-conditions: - User has selected and added products to the cart. - Cart page accessible.		Post-conditions: Cart shows correct product name, price, quantity, and options.		
Description				
Step	Action	Expected System Response	Pass/Fail	Comments
1	Add a product to the cart.	Product successfully added to cart.	Pass	Product added with correct details.
2	Open cart page.	Cart displays product name, quantity, and price correctly.	Pass	Correct product information displayed.

3	Proceed to checkout.	Order summary matches cart details.	Pass	Summary matches cart contents.
---	----------------------	-------------------------------------	------	--------------------------------

5.3.2. Back-end

Automation Testing Test Cases

Table 3.10: Test Case 7 Information

Test Case 7: API Response for Product Recommendation				
System: AI Shopping System			Subsystem: Recommendation Engine	
Designed by: Mr. Huỳnh Hữu Nghĩa			Design Date: 26/04/2025	
Executed by: Mr. Nguyễn Thành Nam			Execution Date: 03/05/2025	
Short Description: Verify that the recommendation API returns valid, relevant product suggestions based on user history				
System Testing: Functional Testing				
Pre-conditions: - User has browsing and purchase history - Recommendation API is live			Post-conditions: JSON response contains relevant product suggestions	
Description				
Step	Action	Expected System Response	Pass/Fail	Comments
1	Send GET request to /api/recommend/{userId}	Response status code 200	Pass	API reachable
2	Check response format	JSON response with product objects	Pass	Schema valid
3	Validate recommendation relevance	Returned items align with user's interests	Pass	Matched based on purchase history

Table 3.11: Test Case 8 Information

Test Case 8: User Authentication Token Validation	
System: AI Shopping System	Subsystem: Authentication
Designed by: Mr.Đào Hữu Hoài	Design Date: 16/04/2025
Executed by: Mr. Huỳnh Hữu Nghĩa	Execution Date: 03/05/2025
Short Description: Ensure valid and expired tokens are handled correctly	
System Testing: Security Testing	
Pre-conditions: - User is registered - Token issued during login	Post-conditions: Token validated or rejected correctly

Description				
Step	Action	Expected System Response	Pass/Fail	Comments
1	Send request with valid token	Access granted (200 OK)	Pass	Token accepted
2	Send request with expired token	Response with 401 Unauthorized	Pass	Token rejected
3	Send request without token	Response with 403 Forbidden or 401	Pass	Access blocked

Table 3.12: Test Case 9 Information

Test Case 9: User Authentication Token Validation				
System: AI Shopping System		Subsystem: Payment Gateway Integration		
Designed by: Mr. Lê Thiên Doanh		Design Date: 15/04/2025		
Executed by: Mr. Nguyễn Thành Nam		Execution Date: 03/05/2025		
Short Description: Test backend payment processing via API				
System Testing: Integration Testing				
Pre-conditions: • Cart is populated • Valid payment method is selected		Post-conditions: • Transaction status recorded in database		
Description				
Step	Action	Expected System Response	Pass/Fail	Comments
1	Send POST to /api/checkout/pay	200 OK and response with transactionId	Pass	Payment API responsive
2	Verify database entry for transaction	Database has matching payment status and amount	Pass	Transaction recorded successfully
3	Send duplicate payment request	Error 409 or duplicate check response	Pass	Proper idempotency in payment

Manual Testing Test Cases

Table 3.13: Test Case 10 Information

Test Case 10: API Failure Handling – Product Detail Fetch	
System: AI Shopping System	Subsystem: Product API
Designed by: Mr. Lê Thiên Doanh	Design Date: 15/04/2025
Executed by: Mr.Đào Hữu Hoài	Execution Date: 03/05/2025
Short Description: Manually test backend response on invalid product ID	

System Testing: Negative Testing				
Pre-conditions: Product ID does not exist			Post-conditions: Error message is returned	
<u>Description</u>				
Step	Action	Expected System Response	Pass/Fail	Comments
1	Call product API with invalid ID	404 Not Found or error message shown	Pass	Handled gracefully
2	Check response format	JSON with error field	Pass	Error structure is correct
3	Retry with valid ID	Product details returned successfully	Pass	Recovers from error input

Table 3.14: Test Case 11 Information

Test Case 11: Admin Dashboard – Add Product Function				
System: AI Shopping System			Subsystem: Product Management	
Designed by: Mr. Huỳnh Hữu Nghĩa			Design Date: 17/04/2025	
Executed by: Mr. Nguyễn Thành Nam			Execution Date: 03/05/2025	
Short Description: Verify that the admin can add new jewelry products successfully				
System Testing: Functional Testing				
Pre-conditions: - Admin is logged in - Product data ready for entry			Post-conditions: Product added and visible to users	
Description				
Step	Action	Expected System Response	Pass/Fail	Comments
1	Navigate to “Add Product”	Form loads with input fields	Pass	Form loads correctly
2	Enter product details	System accepts input and validates data	Pass	All fields validated
3	Submit form	Product appears in product list	Pass	Product saved to database

Table 3.15: Test Case 12 Information

Test Case 12: Payment Integration Flow	
System: AI Shopping System	Subsystem: Payment Services
Designed by: Mr. Nguyễn Thành Nam	Design Date: 16/04/2025
Executed by: Mr. Lê Thiên Doanh	Execution Date: 03/05/2025

Short Description: Test the payment processing flow with external payment gateways (MoMo and ZaloPay).				
System Testing: External Integration Testing				
Pre-conditions: - Order status is "processing". - Payment record exists with isPaid = false. - Gateway test environment is configured			Post-conditions: - Payment marked as paid with timestamp - Order status updated accordingly	
<u>Description</u>				
Step	Action	Expected System Response	Pass/Fail	Comments
1	Send POST request to /api/payments/momo/{orderID}/transaction.	System returns payment URL	Pass	Payment initiation successful.
2	Complete test payment in MoMo sandbox	Callback received with transaction details	Pass	External flow successful
3	Verify system state after payment callback.	isPaid = true, payment date set	Pass	Database updated correctly
4	Repeat tests with ZaloPay payment gateway.	ZaloPay flow works similarly	Pass	Multi-gateway support verified
5	Confirm order status update	Order status = "completed" or appropriate value	Pass	Payment-order sync validated

5.4. Comparison

The AI Jewelry Shopping System offers several advantages over traditional jewelry e-commerce platforms. While conventional systems rely on basic filtering and static recommendations, our system uses AI to provide personalized product suggestions and real-time chatbot assistance. This enhances user experience and engagement. Additionally, automated backend testing and a responsive ReactJS frontend ensure better reliability and usability compared to platforms with outdated interfaces and limited testing.

5.5. Evaluation

The system was evaluated in terms of functionality, usability, performance, and reliability. All core features functioned correctly in both manual and automated tests. Users found the interface intuitive and appreciated the AI-driven filtering and chatbot support. Performance tests showed stable response times under load, and backend automation confirmed system robustness. Overall, the system delivers a reliable, personalized, and efficient jewelry shopping experience.

CHAPTER 6

CONCLUSION AND FEATURE WORK

6.1 Conclusion

In conclusion, the AI-Powered Jewelry Shopping System has successfully delivered a smart, web-based platform that modernizes the jewelry shopping experience through artificial intelligence and immersive visualization. Key features—including AI-driven recommendations, chatbot support, 3D customization—enhance user engagement, convenience, and purchasing confidence.

The system overcomes technical challenges while meeting customer requirements, with highlights such as virtual ring fitting, dynamic pricing, and secure payments, creating an efficient and user-centric shopping environment. Notable achievements include a scalable architecture, adaptive AI modules, and a user-friendly interface that blends innovation with accessibility.

By reducing purchase uncertainty through real-time visualization, the system improves customer satisfaction and lowers return rates. Beyond its technical accomplishments, the project offers valuable insights for future improvements, marking a significant step toward intelligent, personalized, and immersive e-commerce in the jewelry industry.

6.2 Future work

- **Voice-Enabled Shopping:** Integrate voice search and voice command features to improve accessibility and hands-free navigation, especially useful for mobile users or visually impaired customers.
- **Enhanced 3D Customization Interface:** Refine the interactive 3D jewelry customization tool with improved hand gesture tracking, smoother UI transitions, and real-time material rendering for a more immersive design experience.

- **Social Media Integration:** Enable users to share their customized jewelry designs, wishlists, or gift ideas directly on social media platforms to drive organic marketing and user-generated engagement.
- **AR Virtual Try-On:** Expand the virtual fitting feature with Augmented Reality (AR) capabilities, allowing users to try on rings and other jewelry using their smartphone cameras in real-time.
- **Real-Time Price Estimator Enhancements:** Upgrade the automated pricing engine to incorporate dynamic market rates and material availability, ensuring accuracy and transparency in cost estimation.
- **Customer Style Profile:** Introduce user style profiles that evolve over time with browsing and purchase behavior, allowing deeper personalization and more relevant product suggestions.

REFERENCES

1. JON DUCKETT, "HTML and CSS: Design and Build Websites," 2011.
2. JON DUCKETT, "JavaScript and jQuery: Interactive Front-End Web Development," 2014.
3. ALEX BANKS, EVE PORCELLO, "Learning React: Modern Patterns for Developing React Apps," 2020.
4. KINSTA®, "What Is Express.js? Everything You Should Know," n.d.
5. ETHAN BROWN, "Learning Node.js Development," 2019.
6. GILLIS, A., "What is REST API (RESTful API)," 2024.
7. PAUL DUBOIS, "MySQL Cookbook: Solutions for Database Developers and Administrators," 2020.
8. TONY PARISI, "Learning Three.js: The JavaScript 3D Library for WebGL," 2014.
9. DIEGO CANTOR, SACHIN PAVAN, "WebGL Beginner's Guide," 2012.
10. TANMAY BAKSHI, "AI and Machine Learning for Coders," 2021.
11. VANDANA SINGH, "Natural Language Processing for Chatbots," 2020.
12. JASON D. BROWN, "Building Chatbots with Python," 2019.

APPENDIX

Appendix A. Security Measures

1. Authentication

- JWT-based authentication for secure API access.
- Password hashing using bcrypt for enhanced user security.

2. Authorization

- Role-based access control (RBAC) to differentiate between customers, administrators, and manufacturers.
- Middleware verification to restrict access based on user roles.

3. Data Protection

- End-to-end encryption of sensitive customer data, design files, and payment details.
- Regular vulnerability scanning and penetration testing.

4. Payment Security

- Integration with PCI-DSS compliant payment gateways (e.g., Stripe, PayPal).
- Tokenization of credit card information during transactions.

The AI-powered jewelry shopping system applies a comprehensive set of security strategies to ensure the confidentiality, integrity, and availability of user data throughout the shopping journey.

Authentication: To protect communication between the frontend and backend services, JWT (JSON Web Token) authentication is implemented. Each API request is verified using secure tokens, and user credentials are hashed with bcrypt to prevent data leaks in the event of a breach.

Authorization: RBAC is employed to allocate privileges appropriately among different system users, including shoppers, system admins, and third-party jewelers. Middleware mechanisms validate user roles before granting access to protected routes, ensuring that only authorized personnel can access specific functionalities such as order processing or product management.

Data Protection: All sensitive information—ranging from customer profiles to customized design previews and billing details—is encrypted both at rest and during transmission. Security patches and code reviews are regularly conducted to prevent exploits and ensure compliance with industry standards.

Payment Security: By partnering with leading payment service providers that meet PCI-DSS standards, the system ensures secure and trusted financial transactions. Tokenization techniques are used to protect credit card data, further minimizing the risk of fraud.

Appendix B. User Feedback Analysis

1. Positive Feedback Highlights

- "The AI recommendations feel personalized and spot-on."
- "Efficient voice command integration makes shopping more convenient."

2. Negative Feedback Highlights

- "The AI sometimes suggests unrelated products."
- "Takes longer to load when switching between categories."

3. Action Items Based on Feedback

- Refine recommendation engine for better accuracy and relevance.
- Optimize system performance to reduce load time across categories.
- Enhance AI understanding of user preferences through behavioral data and feedback loops.

The AI Shopping System has received strong positive feedback for its smart and personalized shopping experience. Users particularly appreciated the accuracy of the product suggestions and the convenience brought by voice command features, which significantly streamline the browsing and shopping process.

However, there are concerns regarding occasional mismatches in product recommendations, where the AI offers suggestions that don't align with the user's interests. This issue can lead to user frustration and reduce trust in the system's intelligence. Additionally, some users reported delays when navigating through different product categories, affecting overall satisfaction with the speed and responsiveness of the system.

To address these issues, it is essential to fine-tune the recommendation engine by analyzing user behavior more deeply, incorporating advanced machine learning models, and continuously updating data

inputs. Improving system responsiveness requires backend optimization and performance testing to reduce latency during category transitions. Regular updates focusing on AI accuracy, speed, and user satisfaction will be key to maintaining a reliable and enjoyable AI shopping experience. These updates should also include UI improvements, crash prevention, and enhanced system learning from feedback to ensure continuous enhancement of the user journey.

Appendix C. Meeting Minutes

This appendix provides some significant additional meeting minutes that we took note during the process to ensure the work quality.

Meeting Minute 1: Kick-off Meeting (07/02/2025)	
Attendees: All members	Platform: Google Meet
Agenda: + Do requirement analysis + Overview of project goals and objectives + Role assignments + Initial timeline and milestones	
Meeting Minute 2: Sprint Retrospective (26/02/2025)	
Attendees: Thành Nam, Hữu Hoài, Thiên Doanh	Platform: Google Meet
Agenda: + Progress assessment + Addressing any challenges faced + Adjusting timeline if necessary	
Meeting Minute 3: Midterm review (24/03/2025)	
Attendees: All members	Platform: Google Meet
Agenda: + Progress assessment + Addressing any challenges faced + Adjusting timeline if necessary	

Appendix D. Contact Information

For further information or questions regarding the AI Shopping System, please contact:

Project Manager:

- **Name:** Nguyễn Thành Nam
- **Email:** ntnam.work@gmail.com
- **Phone:** (+84) 368 075 139